

Pose Estimation using Flash LiDAR

A project report submitted
in partial fulfillment for the award of the degree of

Bachelor of Technology

in

Electronics and Communication Engineering

by

Samedh Sachin Kari



Department of Avionics Department
Indian Institute of Space Science and Technology
Thiruvananthapuram, India

May 2025

Abstract

This project explores pose estimation techniques using Flash LiDAR, with the objective of determining the position, orientation, and motion states of a target object in three-dimensional space. Building upon prior work in mathematical modeling and preliminary data processing of LiDAR systems, this study develops a framework that enables a chaser system equipped with Flash LiDAR to estimate the pose of a moving target using point cloud data.

The methodology includes the design and implementation of filtering algorithms such as Kalman filter and low-pass filter to reduce noise, computation of relative linear and angular velocities, and the development of a state-based simulation algorithm to facilitate detection, tracking and control. The results demonstrate the robustness of the pose estimation algorithm, with potential applications requiring precise motion tracking, such as spacecraft docking.

Contents

List of Figures	v
List of Algorithms	vii
1 Introduction	1
1.1 Overview of Prior Work	1
1.2 Essence of Pose Estimation	2
1.3 Objectives of the Project	3
1.4 Structure of the Report	3
2 Fundamentals of LiDAR and Pose	4
2.1 LiDAR	4
2.2 Pose	5
3 Rigid Body Kinematics and Dynamics	7
3.1 Rotational Kinematics	7
3.2 Rotational Dynamics	11
3.3 Euler's Equation	12
4 Pose Estimation Algorithm	14
4.1 Iterative Closest Point Algorithm	14
4.2 The Kalman Filter	19
4.3 Filtering Orientation Estimates	23
4.4 Pose Estimation Algorithm	24
4.5 State-based Sensor Workflow	26
5 Implementation and Results	32
5.1 Rigid Body Simulations	32
5.2 Reference and Target Images for ICP	33
5.3 Kalman Filter Implementation	36
5.4 Low Pass Filter Results	37

5.5	Sensor Workflow Implementation	37
6	Conclusion and Future Work	43
6.1	Limitations	43
6.2	Future Possibilities	44
	Bibliography	44
	Appendices	47
A	C++ Programs	47
A.1	Matrix Library	47
A.2	Rigid Library	60
A.3	Pose Library	68
A.4	20000 Second Simulation	73
A.5	Pose Estimation	74
A.6	Sensor Workflow	78

List of Figures

1.1	A Chaser Target System	2
2.1	Flash LiDAR	4
2.2	Pose as Position and Orientation	5
2.3	Frames of Reference	6
3.1	Rotation of a Rigid Body	11
4.1	An Ideal Case	14
4.2	Iterative Closest Point	17
4.3	Prediction Correction Method	21
4.4	Kalman Filter	23
4.5	Bode Plot for Low Pass Filter	25
4.6	Pose Estimation Algorithm	26
4.7	Sensor Workflow	27
4.8	Sensor Workflow	31
5.1	Components of $\vec{\omega}$ for 20000 seconds	33
5.2	Euler Angles for 20000 seconds	34
5.3	Euler Angles for 4000 seconds	35
5.4	Reference LiDAR Image	36
5.5	Kalman Position Results	38
5.6	Kalman Velocity Results	39
5.7	Low Pass Filter Angles Results	40
5.8	Low Pass Filter Velocity Results	41
5.9	Sensor Workflow Results	42

List of Algorithms

1	Point Cloud Registration	17
2	Iterative Closest Point	18

Chapter 1

Introduction

This report documents the work carried out as part of my B.Tech project, which is a continuation of my internship titled 'Mathematical Modeling and Preliminary Data Processing of LiDAR'. The primary focus of this project is to develop methods for pose estimation using Flash LiDAR technology. This report presents the methodologies, algorithms, and results related to pose estimation using Flash LiDAR data, extending the work initiated during the internship.

1.1 Overview of Prior Work

The internship titled 'Mathematical Modeling and Preliminary Data Processing of LiDAR' focused mainly on two objectives, which were foundational steps towards the current project.

1. **Mathematical Modeling:** This part involved a component-wise analysis of the working principles of LiDAR, specifically focusing on Flash LiDAR. It included modeling the laser source, an optically grated surface used to create a flash, reflection characteristics at the target, and the array of photodiodes that capture the reflected signal. This model was simulated using the Finite Difference Time Domain (FDTD) method.
2. **Preliminary Data Processing:** The second focus of the internship was on processing LiDAR point-cloud data. This part involved developing a noise removal technique based on an original clustering algorithm. This clustering method was designed to group LiDAR points based on spatial closeness, which has been used to process images in the current project.

With the mentioned objectives, the internship laid a foundation through mathematical modeling of Flash LiDAR and the development of a noise removal algorithm, both of which are crucial for the successful implementation of pose estimation techniques in this project.

1.2 Essence of Pose Estimation

Pose estimation, like many modern algorithms, finds its inspiration in nature, particularly in the mechanisms by which the human brain interprets visual information. Upon capturing an image, the brain identifies various (rigid) objects within the scene, along with their respective positions and orientations. This process occurs continuously and subconsciously, but it becomes critical during movement, where tasks such as avoiding obstacles, moving through unknown terrain, and chasing a target are performed.

In order to enable autonomous movement in machines, it is necessary to first have a sensor capable of imaging, followed by an algorithm that can determine the position and orientation of objects within the captured image, just like the functionality of the brain. This project is a step toward the development of such an image processing algorithm.

The pose estimation framework developed in this project is mainly applicable to any target-chaser system where the chaser is equipped with an imaging sensor to observe the target. In this case, the sensor is a Flash LiDAR, which captures a three-dimensional point cloud image of the target object. However, the proposed methods can be extended to any sensor capable of three-dimensional imaging. The chaser is assumed to retain a reference image of the target in its own memory.

Pose estimation compares the captured image with the stored reference image to determine the transformation between the two frames of reference. Since the chaser and target may be in motion, this transformation must account for changes in both position and orientation over time. When time-dependent sensor data is available, the relative linear and angular velocities become more reliable measures of motion of the chaser and the target. These can be computed by differentiating the available pose data.

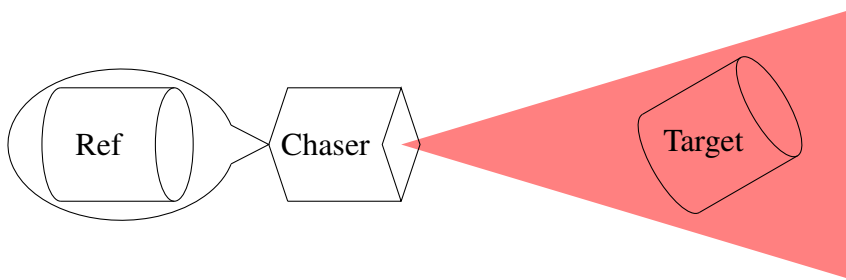


Figure 1.1: A Chaser Target System

The estimated pose and velocity data can be supplied to a control algorithm on-board the chaser, which then computes and applies the required force and torque inputs to align or pursue the target effectively. This is the fundamental idea behind pose estimation. It finds critical applications in various real-world scenarios, such as spacecraft docking operations.

1.3 Objectives of the Project

The primary focus of this project is ‘Pose Estimation using Flash LiDAR’, which involves determining the pose of an object using the three-dimensional point cloud images obtained from a Flash LiDAR sensor. The states estimated as part of the pose primarily include position and orientation, but this project extends them to include linear and angular velocities.

1. Estimation of the position and orientation of a moving target with respect to a chaser using point cloud data obtained from a Flash LiDAR sensor.
2. Design and implementation of filtering algorithms to reduce the noise inherent in pose estimation.
3. Computation of relative linear and angular velocities from time-dependent pose data.
4. Design of a preliminary simulation algorithm with a state-based logic flow that allows the chaser to detect, track, and lock on to the target autonomously.

1.4 Structure of the Report

This report is structured in 5 chapters. Chapter 2 introduces the Flash LiDAR sensor and the concept of pose, focusing on considerations related to frame of reference necessary for pose estimation. Chapter 3 covers the fundamentals of rigid body kinematics and dynamics, which are essential for pose estimation, since the target is a rigid body. Chapter 4 delves into the theoretical foundations of the pose estimation algorithm, discussing the Iterative Closest Point Algorithm and the Kalman Filter. Finally, Chapter 5 presents the implementation of the methods outlined in Chapter 4, followed by a discussion of the results and their implications. Chapter 6 concludes the report and highlights future possibilities to extend the project.

Chapter 2

Fundamentals of LiDAR and Pose

Knowing the fundamentals related to LiDAR and pose is a prerequisite to work on pose estimation using flash LiDAR. This chapter explores the fundamental concepts of LiDAR technology and the principles of pose estimation that are essential for pose estimation.

2.1 LiDAR

LiDAR, an acronym for Light Detection and Ranging, is a remote sensing technology that utilizes laser pulses to determine distances to a target, by measuring the time of flight of the laser beam.

2.1.1 Flash LiDAR

Flash LiDAR is one of the types of LiDAR, which illuminates the field of view by creating a flash. A flash has coherent and monochromatic light diffused by a grated surface and its reflections are captured by a 2D array of photo detectors that record the time of flight. This time of flight data is converted to a point cloud image by considering the specifications of the optical elements present in the LiDAR.

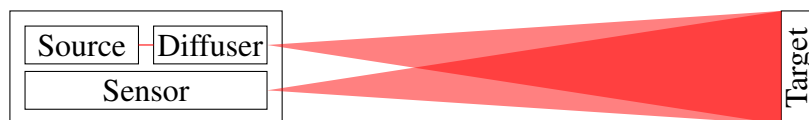


Figure 2.1: Flash LiDAR

2.1.2 Point Cloud Images

LiDAR images are point cloud images. As discussed previously, the light sensed by every photodetector is converted to 3 dimensional coordinates of the points. Hence,

the final image is a point cloud, which is a set of points in 3 dimensions. The point cloud represents the imaged surface sampled in x and y directions with respect to the sensor. Several models of flash LiDAR also provide the intensity data, which can be used to identify the relative reflectances of the surfaces. The target in the reference and target in the algorithm refer to the point cloud images of the actual reference and target objects, which are expressed with respect to their centroid or point of interest.

2.2 Pose

The term ‘pose’ refers to the position and orientation of an object in the 3D space. The primary requirement to describe any form of position or orientation is a reference frame. Hence, a pose can be mathematically described with reference to a coordinate system. Once the coordinate system is established, a pose is any coordinate transformation that involves rotation and translation, but not scaling. This can be expressed as a translation vector $\vec{r} = x\hat{i} + y\hat{j} + z\hat{k}$ and relative orientation, which is denoted using rotation angles ϕ, θ, ψ about the x, y and z axes, respectively, or quaternions or direction cosine matrix, as shown in Figure 2.2.

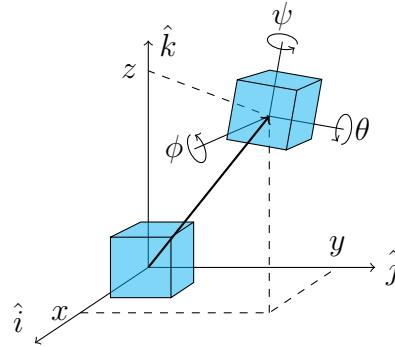


Figure 2.2: Pose as Position and Orientation

2.2.1 Frames of Reference

It is easy to describe pose in a particular frame of reference, but the pose data can be practically used only if the kinematics and dynamics of the frame are known with respect to an inertial frame of reference. However, in our case, there will be 3 frames of reference. These include an inertial frame of reference, and two non-inertial body fixed frames fixed to the sensor and the target.

The pose estimation performed will find the position and orientation of the target with respect to the sensor, which is the relative transformation between the target frame

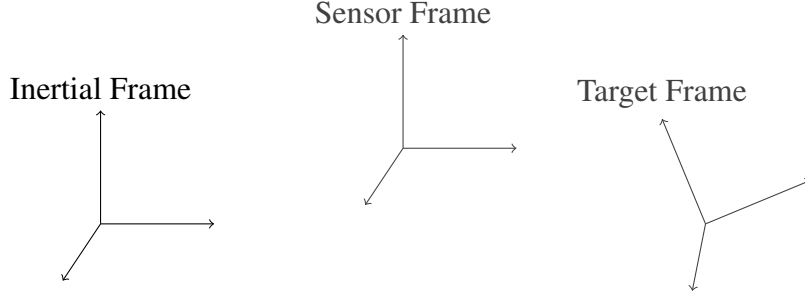


Figure 2.3: Frames of Reference

and the sensor frame. The pose is then converted to an absolute frame, by using the absolute pose of the sensor (which is measured using an Inertial Measurement Unit). This will be quite straightforward for the position, as position can be expressed as vectors.

$$\vec{r}_{TI} = \vec{r}_{TS} + \vec{r}_{SI}$$

In the above expression, I , S and T stand for inertial, sensor, and target. $\vec{r}_{TS}$ represents the position of the target frame with respect to the sensor frame.

On the other hand, orientation is not commutative because two rotations in different sequences lead to different orientations. Hence, orientation is not a vector quantity and needs separate treatment. It can be handled by multiplying the rotation matrices or quaternions relative to the frames in a particular sequence to obtain the desired rotation.

$$\mathbf{q}_{TI} = \mathbf{q}_{SI} \otimes \mathbf{q}_{TS}$$

$\otimes$ represents quaternion product defined in Section 3.1.2. Additionally, all vectors are defined in the inertial frame of reference, and to use them in any other frame, they must be transformed accordingly. Therefore, the inertial frame serves as the fundamental reference frame for simulation, with other frames derived by applying transformations to the vectors.

Chapter 3

Rigid Body Kinematics and Dynamics

In the context of pose estimation, the target is typically considered as a rigid body. The standard definition of a rigid body is a collection of particles, where the distance between any two particles remains constant over time irrespective of time. This chapter delves deeper into rigid body kinematics and dynamics and modeling the target as a rigid body.

Newtonian mechanics does a great job analyzing kinematics and dynamics, but all the basic quantities like momentum are defined for point masses. Practically, there are more complex objects that are not point masses, and these need to be handled by considering infinite point masses that have infinitesimal mass dm . For some objects, it is always possible to set up a coordinate system in which the object performs purely rotational motion. In other words, if an object is made up of points of mass dm , then every point will perform a purely rotational motion (the distance from the origin does not change) and the angular velocity, $\vec{\omega}$, for every point will be equal. Such objects can be called rigid bodies. Since all points rotate with the same $\vec{\omega}$, any two points will naturally have the same relative position regardless of time. The analysis of kinematics and dynamics of a rigid body can be done in two parts. First, the kinematics and dynamics of the new coordinate system and, second, rotational motion about the origin of the new coordinate system. In this report, the target whose pose is to be estimated is assumed to be a rigid body, and the origin of the new coordinate system will be called the **point of interest**.

3.1 Rotational Kinematics

Rigid body kinematics and dynamics is divided into two parts, first, the kinematics and dynamics of a point of interest, and second, being a purely rotational motion about the point of interest. This makes rotation of vectors a critical operation. Furthermore, as discussed in the previous chapter, the angle of rotation, θ , cannot be treated as a vector quantity because it is not commutative. This section explores several mathematical

strategies used to express the rotation of vectors.

3.1.1 Euler Angles

Euler angles are a sequence of three rotations about the coordinate axes that are capable of achieving all rotation. Mathematically, rotation of a vector about the coordinate axes can be performed by multiplying the vector by the corresponding rotation matrices.

$$R_x(\psi) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \psi & \sin \psi \\ 0 & -\sin \psi & \cos \psi \end{bmatrix}, R_y(\theta) = \begin{bmatrix} \cos \theta & 0 & -\sin \theta \\ 0 & 1 & 0 \\ \sin \theta & 0 & \cos \theta \end{bmatrix}, R_z(\phi) = \begin{bmatrix} \cos \phi & \sin \phi & 0 \\ -\sin \phi & \cos \phi & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Intuitively, if we have just one vector in two different orientations, then a single rotation along a mutually perpendicular axis is sufficient to match the orientation of the two vectors. However, if we have two sets of three orthogonal axes in different orientations and only rotation about these axes is allowed, then to match one of the axes, we would need two rotations. First, a rotation about the axis under consideration to get a different axis along the cross product of the two orientations of the axis under consideration. Second, a rotation about the axis along the cross product to match the axis under consideration. Hence, for any two orientations, we can match one axis using two rotations. The third rotation about the matched axis would match the other two coordinate systems.

In the above interpretation, the axis of rotation for the first and third rotations are the same. This is how Euler worked it out and introduced the sequences $z-x-z, x-y-x, y-z-y, z-y-z, x-z-x, y-x-y$ for rotation. Later, Tait–Bryan generalized this concept for the case where the first and third angles are different, adding more possibilities to rotation sequences such as $x-y-z, y-z-x, z-x-y, x-z-y, z-y-x, y-x-z$.

3.1.2 Quaternions

The idea of quaternions comes from complex numbers. Fundamentally, complex numbers are a combination of real and imaginary numbers, where imaginary numbers are defined as $\sqrt{-1} = i$. However, $i^4 = 1$, which makes the powers of i periodic with period 4. These complex numbers can be plotted on the 2D Cartesian plane where the x axis is the real axis and the y axis is the imaginary axis, and the periodicity of the powers of i can be associated with rotation. In this system, a 2-dimensional vector can be expressed as a complex number and multiplying this with a complex number of unit

norm will rotate the vector without scaling it. This means that all the complex numbers lying on the unit circle correspond to a particular rotation. It can be shown that the angle of rotation is the angle that the complex number makes with the positive real axis.

To extend this idea to higher dimensions, we will need complex numbers of higher dimensions. In my opinion, the most attractive way to create a complex number of higher dimensions is the Cayley-Dickson construction. It defines a four-dimensional complex number that can be defined as

$$\mathbf{q} = A + Cj = (a + bi) + (c + di)j$$

where A and C are two-dimensional complex numbers and $ij \doteq k$. Such numbers are called quaternions and can be expressed as

$$\mathbf{q} = q_0 + q_x i + q_y j + q_z k$$

where i, j and k are governed by

$$i^2 = j^2 = k^2 = ijk = -1$$

The addition and subtraction of quaternions is quite straightforward, but multiplication becomes interesting because the product of quaternions is not commutative. The multiplication of quaternions can be mathematically simplified as

$$\begin{aligned} \mathbf{p} \otimes \mathbf{q} &= (p_0 + p_x i + p_y j + p_z k) (q_0 + q_x i + q_y j + q_z k) \\ &= \begin{bmatrix} p_0 & -p_x & -p_y & -p_z \\ p_x & p_0 & -p_z & p_y \\ p_z & p_z & p_0 & -p_x \\ p_z & -p_y & p_x & p_0 \end{bmatrix} \begin{pmatrix} q_0 \\ q_x \\ q_y \\ q_z \end{pmatrix} \\ &= \begin{bmatrix} q_0 & -q_x & -q_y & -q_z \\ q_x & q_0 & q_z & -q_y \\ q_z & -q_z & q_0 & q_x \\ q_z & q_y & -q_x & q_0 \end{bmatrix} \begin{pmatrix} p_0 \\ p_x \\ p_y \\ p_z \end{pmatrix} \end{aligned} \quad (3.1)$$

A four-dimensional vector can be transformed by multiplying it with a quaternion, and to avoid any effect of scaling in this operation, the quaternion should have unit norm just like the complex numbers.

$$q_0^2 + q_x^2 + q_y^2 + q_z^2 = 1$$

If plotted in a four-dimensional space, this quaternion should lie on a four-dimensional

sphere of unit radius. We can express the quaternion using $q_0 = \cos \theta$ and $q_x i + q_y j + q_z k = \vec{u} \sin \theta$, where $\vec{u}$ is a unit vector. By substituting this in the matrix we can get a feel that this quaternion would rotate the four-dimensional vector by an angle of θ . However, practically we need to work with three-dimensional vectors. We express a three-dimensional vector as a pure quaternion whose real part, $q_0 = 0$. When transforming a three-dimensional vector using quaternions, it should not be pulled out of the three-dimensional space (i.e. q_0 should remain 0). To ensure this, the triple product of quaternions is defined as

$$L_q = \mathbf{q} \circledast \mathbf{p} \circledast \mathbf{q}^*$$

where $\mathbf{p} = 0 + \vec{p}$, $\mathbf{q} = \cos(\theta/2) + \vec{u} \sin(\theta/2)$ and $\mathbf{q}^* = \cos(\theta/2) - \vec{u} \sin(\theta/2)$ for rotation by an angle of θ about the unit vector $\vec{u}$.

The derivative of a time-varying quaternion is given by the quaternion product of the quaternion itself and the angular velocity vector in three dimensions expressed as a quaternion.

$$\frac{d\mathbf{q}(t)}{dt} = \frac{1}{2} \mathbf{q} \circledast \boldsymbol{\omega}_{\text{rel}} \quad (3.2)$$

where $\boldsymbol{\omega}_{\text{rel}} = 0 + \vec{\omega}_{\text{rel}}$, which is the angular velocity expressed in frame relative to the rigid body.

Angular velocity, $\vec{\omega}$, is the differentiation of orientation with respect to time. However, $\vec{\omega}$ is a vector unlike rotation matrices or quaternions. Orientation could not be a vector for not being commutative, but angular velocity is a vector because a sum of finite infinitesimal rotations can be commutative.

3.1.3 Angular Velocity in Relative Frames

Equation 3.2 raises an obvious question. Recalling the discussion in Section 2.2.1, in which reference frame do we consider $\vec{\omega}$? However, the answer lies in Section 2.2.1 itself.

Equation 3.2 actually makes sense. For time being, considering just one absolute frame and no other frames of reference, the differentiation of quaternions is actually the limiting case for the ratio of the difference between quaternions expressing rotations at a very small time gap to the time delay. As quaternions are not commutative, the quaternion at the next instant will be the product of quaternion corresponding to infinitesimal rotation and the quaternion at current instant. The limiting case for the quaternion corresponding to infinitesimal rotation divided by time delay gives $\vec{\omega}$. So,

the differentiation of quaternion is

$$\frac{d\mathbf{q}(t)}{dt} = \frac{1}{2}\boldsymbol{\omega}_{\text{abs}} \circledast \mathbf{q}$$

$\boldsymbol{\omega}_{\text{abs}} = 0 + \vec{\omega}$, $\vec{\omega}$ is constant and independent of the frame, and different frames are just different ways of viewing the same vector. We want to choose the way of viewing $\vec{\omega}$ that is computationally most convenient to us. Since we are dealing with multiple frames of reference, considering $\vec{\omega}$ in the local frame gives the quaternion in the local frame, which makes it easier to interpret rotation. This shift in frames is accommodated by the change in the sequence of multiplication. However, the key idea here is that when a rigid body rotates with an angular velocity $\vec{\omega}$, the value of $\vec{\omega}$ is constant, and different frames of reference simply represent different perspectives on the same rotation.

3.2 Rotational Dynamics

It is important to note that a new coordinate system has been set here and this will have some consequences. After position ($\vec{r}$) itself, the next quantity that depends on the coordinate system is the angular momentum, which is defined as

$$\vec{h} = \vec{r} \times m\vec{v} \quad (3.3)$$

During analysis of the linear momentum $\vec{p} = m\vec{v}$, or the angular momentum of a point mass $\vec{h} = m\|\vec{r}\|^2\vec{\omega}$, the momentum and the respective velocity always have the same direction. This relationship can be extended to the fact that velocity and momentum are linearly related. Any scaling or combination of velocities results in a proportional change in momentum.

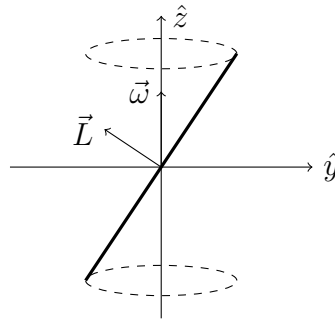


Figure 3.1: Rotation of a Rigid Body

However, in the case of rigid bodies, the relation between $\vec{\omega}$ and $\vec{h}$ is not straightforward. For example, consider a tilted rod (rigid body) that is rotating about the $\hat{z}$ axis as seen in Figure 3.1. The angular velocity for this rod is only in the $\hat{z}$ direction, but the

angular momentum has components in the $\hat{y}$ and $\hat{z}$ directions. Hence, changing the $\hat{z}$ component of the angular velocity will linearly change the $\hat{y}$ and $\hat{z}$ components of the angular momentum. Changing one component of the angular velocity can change all components of the angular velocity [1].

We need to map the angular velocity vector to the angular momentum vector such that a single component of $\vec{\omega}$ contributes to every component of $\vec{h}$. A tensor, which can be expressed as a matrix, is a mathematical element capable of mapping one vector to another, and can be used to express the moment of inertia. An expanded form of the equation $\vec{h} = I\vec{\omega}$ can be expressed as:

$$\begin{pmatrix} h_x \\ h_y \\ h_z \end{pmatrix} = \begin{bmatrix} I_{xx} & I_{xy} & I_{xz} \\ I_{xy} & I_{yy} & I_{yz} \\ I_{xz} & I_{yz} & I_{zz} \end{bmatrix} \begin{pmatrix} \omega_x \\ \omega_y \\ \omega_z \end{pmatrix}$$

3.3 Euler's Equation

For a rigid body, let $\vec{r}$ be any vector towards a point on the rigid body in the inertial coordinate system. The same vector can be expressed as the sum of the position of the point of interest and a vector in the coordinate system centered on the point of reference.

$$\vec{r} = \vec{R} + \vec{r}'$$

where $\vec{R}$ is the vector that points towards the point of interest in the inertial coordinate system and $\vec{r}'$ is the vector in the moving coordinate system centered at the point of interest. On differentiating the above equation in the point of interest centered coordinate system, we get

$$\left. \frac{d\vec{r}}{dt} \right|_{\text{abs}} = \left. \frac{d\vec{r}'}{dt} \right|_{\text{rel}} + \vec{\omega} \times \vec{r}' \quad (3.4)$$

In this equation, $\vec{R}$ is missing because the right side of the equation uses a reference frame centered at $\vec{R}$.

Balancing the torque on the rigid body using $\vec{\tau}_{\text{net}} = \frac{d\vec{h}}{dt}$ and Equation 3.4, the following expression can be obtained, where all terms are in the coordinate frame centered at the point of interest.

$$\tau_{\text{net}} = \left(\frac{d\vec{h}}{dt} \right)_{\text{abs}} = \left(\frac{d\vec{h}}{dt} \right)_{\text{rel}} + \vec{\omega} \times \vec{h}$$

Using the moment of inertia defined as a tensor, we can convert the above equation to its state space form, which is

$$\dot{\vec{\omega}} = I^{-1} (\vec{\tau}_{\text{net}} - \vec{\omega} \times (I\vec{\omega})) \quad (3.5)$$

Chapter 4

Pose Estimation Algorithm

This chapter presents the algorithms and techniques employed in pose estimation using Flash LiDAR. It includes a detailed discussion of the Iterative Closest Point (ICP) algorithm for point cloud alignment, along with filtering methods such as the Kalman filter and low-pass filters for noise reduction. The chapter concludes with an integrated workflow outlining the appropriate conditions for deploying each algorithm, as well as the scheduling logic for their activation within the system.

4.1 Iterative Closest Point Algorithm

The ICP algorithm is the heart of pose estimation. It is an elegant extension of the quaternion estimation algorithm, which is used in star sensors. The quaternion estimation algorithm determines the relative orientation of an image with a reference. The ideal case of the ICP first moves the reference and the image to a common point of interest, and then uses the same logic as quaternion estimation. It further estimates the position by comparing the positions of the centroids. It then develops an iterative approach to address non-ideal situations by generating new targets (which mimic ideal ones) and reduce the mean square error, offering a reliable estimate of the pose.

4.1.1 An Ideal Case

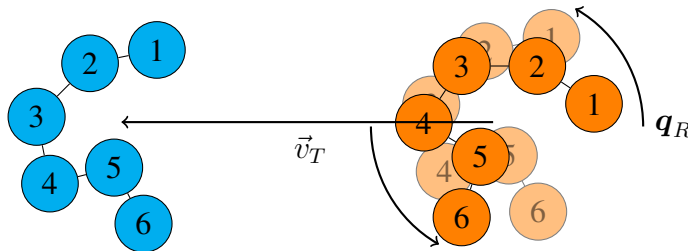


Figure 4.1: An Ideal Case

Consider the simplest case where we have two point clouds, as shown in Figure 4.1. The blue point cloud represents the reference model and the orange represents the target. In this case, both point clouds are identical except for being rotated and translated. This means that both point clouds have the same number of points and the points in both point clouds are indexed, and the indexing for both point clouds also matches.

Let X be the reference point cloud that has points $\vec{x}_i$ and P be the given data with points $\vec{p}_i$, where $1 \leq i \leq n$ which means both X and P have n points. Let P be obtained by rotating X by the quaternion $\mathbf{q}_R$ and translating it by the vector $\vec{v}_T$. The aim is to find $\mathbf{q}_R$ and $\vec{v}_T$ [2]. This can be defining and minimizing the error given by

$$f(\mathbf{q}_R, \vec{v}_T) = \frac{1}{n} \sum_{i=1}^n \left\| \vec{x}_i - L_{\mathbf{q}_R}(\vec{p}_i) - \vec{v}_T \right\|^2$$

where $L_{\mathbf{q}_R}(\vec{p}) = \mathbf{q}_R \vec{p} \mathbf{q}_R^*$ is the vector $\vec{p}$ rotated by the quaternion $\mathbf{q}_R$.

First, we will find $\mathbf{q}_R$ while handling $\vec{v}_T$ by translating both X and P to their centroids $\vec{\mu}_P$ and $\vec{\mu}_X$, respectively, as shown.

$$\vec{\mu}_X = \frac{1}{n} \sum_{i=1}^n \vec{x}_i \quad \text{and} \quad \vec{\mu}_P = \frac{1}{n} \sum_{i=1}^n \vec{p}_i$$

We define $\vec{x}'_i = \vec{x}_i - \vec{\mu}_X$ and $\vec{p}'_i = \vec{p}_i - \vec{\mu}_P$. To accommodate the remaining terms, $\vec{v}'_T = \vec{v}_T - \vec{\mu}_X + L_{\mathbf{q}_R}(\vec{\mu}_P)$, resulting in the modified error function shown in Equation 4.1.

$$f(\mathbf{q}_R, \vec{v}_T) = \frac{1}{n} \sum_{i=1}^n \left\| \vec{x}'_i - L_{\mathbf{q}_R}(\vec{p}'_i) - \vec{v}'_T \right\|^2 \quad (4.1)$$

Expanding this function further, we get

$$f(\mathbf{q}_R, \vec{v}_T) = \sum_{i=1}^n \left\| \vec{x}'_i - L_{\mathbf{q}_R}(\vec{p}'_i) \right\|^2 - 2\vec{v}'_T \cdot \sum_{i=1}^n \left[\vec{x}'_i - L_{\mathbf{q}_R}(\vec{p}'_i) \right] + n \left\| \vec{v}'_T \right\|^2$$

In the above equation, the sum in the middle term is the sum of a set of vectors with respect to their centroid, which will be zero. The optimum $\mathbf{q}_R$ would follow $\vec{v}'_T = \vec{\mu}_X - L_{\mathbf{q}_R}(\vec{\mu}_P)$, which implies $\vec{v}'_T = 0$. Hence, the second and third terms become zero and the optimum $\mathbf{q}_R$ would correspond to the minimum value of the first term in the above equation. Expanding the first term, we get

$$\sum_{i=1}^n \left\| \vec{x}'_i - L_{\mathbf{q}_R}(\vec{p}'_i) \right\|^2 = \sum_{i=1}^n \left\| \vec{x}'_i \right\|^2 - 2 \sum_{i=1}^n \left[\vec{x}'_i \cdot L_{\mathbf{q}_R}(\vec{p}'_i) \right] + \sum_{i=1}^n \left\| L_{\mathbf{q}_R}(\vec{p}'_i) \right\|^2$$

In the above equation, the first and third terms are the sums of distances of points from the centroid, which is independent of $\mathbf{q}_R$. Hence, maximizing the second term would minimize $f(\mathbf{q}_R, \vec{v}_T)$.

$$\begin{aligned}
& \sum_{i=1}^n \left[L_{\mathbf{q}_R}(\vec{p}'_i) \cdot \vec{x}'_i \right] \\
&= \sum_{i=1}^n \left[\mathbf{q}_R \vec{p}'_i \mathbf{q}_R^* \cdot \vec{x}'_i \right] \\
&= \sum_{i=1}^n \left[-\vec{p}'_i \mathbf{q}_R \cdot \vec{x}'_i \mathbf{q}_R \right] \\
&= \sum_{i=1}^n \left[\left(-\vec{p}'_i \mathbf{q}_R \right)^T \left(\vec{x}'_i \mathbf{q}_R \right) \right] \\
&= \sum_{i=1}^n \left[\mathbf{q}_R^T \begin{bmatrix} 0 & p'_{xi} & p'_{yi} & p'_{zi} \\ -p'_{xi} & 0 & -p'_{zi} & p'_{yi} \\ -p'_{yi} & p'_{zi} & 0 & -p'_{xi} \\ -p'_{zi} & -p'_{yi} & p'_{xi} & 0 \end{bmatrix} \begin{bmatrix} 0 & -x'_{xi} & -x'_{yi} & -x'_{zi} \\ x'_{xi} & 0 & x'_{zi} & -x'_{yi} \\ x'_{yi} & -x'_{zi} & 0 & x'_{xi} \\ x'_{zi} & x'_{yi} & x'_{xi} & 0 \end{bmatrix} \mathbf{q}_R \right] \quad (4.2) \\
&= \mathbf{q}_R^T \sum_{i=1}^n \left[\begin{bmatrix} 0 & p'_{xi} & p'_{yi} & p'_{zi} \\ -p'_{xi} & 0 & -p'_{zi} & p'_{yi} \\ -p'_{yi} & p'_{zi} & 0 & -p'_{xi} \\ -p'_{zi} & -p'_{yi} & p'_{xi} & 0 \end{bmatrix} \begin{bmatrix} 0 & -x'_{xi} & -x'_{yi} & -x'_{zi} \\ x'_{xi} & 0 & x'_{zi} & -x'_{yi} \\ x'_{yi} & -x'_{zi} & 0 & x'_{xi} \\ x'_{zi} & x'_{yi} & x'_{xi} & 0 \end{bmatrix} \right] \mathbf{q}_R
\end{aligned}$$

This quaternion product has been expanded as a 4×4 matrix and simplified using Equation 3.1. To maximize the value of this sum, after the sum of matrix products has been calculated for the given point cloud data, the eigenvector corresponding to the largest eigenvalue would give the quaternion corresponding to optimum rotation.

After getting the rotation quaternion, obtaining the translation remains no big deal, as the centroids of the point clouds are already known.

$$\vec{v}_T = \vec{\mu}_X - L_{\mathbf{q}_R}(\vec{\mu}_P)$$

In this case, with two input point clouds P and X , we have obtained the transformation given by $\mathbf{q}_R$ and $\vec{v}_T$ that relates the two point clouds.

Algorithm 1: Point Cloud Registration

Input: Reference Pointcloud X , Target Pointcloud P

Output: $q_R, \vec{v}_T$

- 1 $\vec{\mu}_X = \frac{1}{n} \sum_{i=1}^n \vec{x}_i$, $\vec{\mu}_P = \frac{1}{n} \sum_{i=1}^n \vec{p}_i$
 - 2 $\vec{x}'_i = \vec{x}_i - \vec{\mu}_x$, $\vec{p}'_i = \vec{p}_i - \vec{\mu}_p$
 - 3 $q_R = \arg \min_q \sum_i ||\vec{x}'_i - L_{q_R}(\vec{p}'_i)||^2$
 - 4 $\vec{v}_T = \vec{\mu}_X - L_{q_R}(\vec{\mu}_P)$
 - 5 **return** $q_R, \vec{v}_T$
-

For valid inputs, the results of Algorithm 1 always obey the following.

$$\vec{x}_i = L_{q_R}(\vec{p}_i) + \vec{v}_T \quad \forall \quad 1 \leq i \leq n$$

4.1.2 Iterating for the Closest Point

The previous case works perfectly, but it is too ideal because of the following assumptions.

1. The reference and target point clouds have equal number of points.
2. The points in both point clouds are indexed.
3. Indexing in both point clouds matches.

Figure 4.2 represents the case that avoids the above assumptions. Both point clouds represent the same shape, but the points are not indexed and may have unequal points.

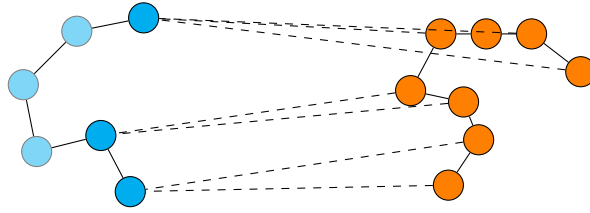


Figure 4.2: Iterative Closest Point

The ICP Algorithm [3] overcomes these assumptions by constructing a new point cloud reference from the given image that has exactly as many points as the image, which are indexed to match the image. This is done by mapping every point in the image to the geometrically closest point in the reference (with replacement of reference points). In other words, for every image point $\vec{p}_i$, a corresponding target point $\vec{y}_i$ is obtained from X such that

$$||\vec{p}_i - \vec{y}_i|| = \min_{\vec{x} \in X} ||\vec{p}_i - \vec{x}||$$

This point cloud is called Y (represented by bright points in the reference). The ICP algorithm can now be stated as

Algorithm 2: Iterative Closest Point

Input: Reference Pointcloud X , Target Pointcloud P , Threshold τ

Output: $q_R, \vec{v}_T$

```

1  $P_0 = P, q_0 = 1, \vec{v}_0 = 0, k = 0$ 
2 while  $e_k - e_{k-1} > \tau$  do
3   for  $\vec{p}_i \in P_k$  do
4      $\vec{y}_i = \arg \min_{\vec{x} \in X} \|\vec{p}_i - \vec{x}\|$ 
5    $(q_k, \vec{v}_k) = \text{Point Cloud Registration}(P_0, Y)$ 
6    $P_{k+1} = L_{q_k}(P_0) + \vec{v}_k$ 
7    $e_k = \frac{1}{N} \sum_i \|\vec{y}_i - L_{q_k}(\vec{p}_i) - \vec{v}_k\|^2$ 
8 return  $q_k, \vec{v}_k$ 

```

4.1.3 Convergence of the ICP Algorithm

The convergence of the ICP algorithm is based on the fact that the error for every iteration is less than that for the previous iteration. This is quite intuitive, as Point Cloud Registration prescribed in Algorithm 1 always returns the transformation corresponding to least error. Furthermore, the base case of every iteration in the ICP algorithm is the result of Point Cloud Registration for the previous iteration. This guarantees that $e_{k+1} < e_k$ will always be true.

This offers a different approach to analyze the convergence of the ICP algorithm. Every iteration performed is similar to a step in gradient descent, where we are moving towards a local minima of the mean square error function.

4.1.4 Limitations of the ICP Algorithm

As seen in Figure 4.2, the ICP algorithm creates a subset of the reference with the repetition of points to obtain rotation and translation using the ideal case. However, since we have many-to-one mapping between the target and reference, the algorithm fails to converge in some cases.

1. Single closest point : If all points of the reference have the same closest point of the target, then in Equation 4.2, one of the matrix will be the same for all points and can be removed from the summation. Since the sum of the second matrix is calculated with respect to the point of interest (which is the centroid), it results in a singularity, causing the algorithm to fail.

2. Repeated closest points : If the same set of points of the reference point cloud is chosen for consecutive iterations, then the error is reduced to 0 ($< \tau$) and the algorithm converges. However, this does not guarantee an optimum orientation and may happen for some other orientation as well. This is similar to finding a local minima of the mean square error function which is not a global minima.

The ICP algorithm is a good workaround for estimating the position and orientation for practical cases. However, many-to-one mapping conditions result in the algorithm not converging. The reason behind these problems is partially physical and partially mathematical. Mathematically, the error function considers only the distance between closest points, which is clearly not sufficient. However, such a situation would arise when the viewing angle is not optimum. Looking at a surface face-on would smoothly converge the algorithm, but if observed edge-on, it would lead to one of these cases. So, physically, adjusting the position of the sensor is one of the solutions to avoid problematic cases.

4.1.5 Improvements

The most obvious improvement in this algorithm would be to consider orientation along with distance in the error function. This could be done by adding a component of $\cos \theta$, where θ is the angle between the surface normals at the corresponding closest points of the reference and the target.

4.2 The Kalman Filter

The ICP Algorithm would give the raw pose estimates, which are often noisy. Reliable results can only be obtained once this noise is mitigated. Given the linear nature of position dynamics, the Kalman filter is the most suitable technique for filtering the position data.

4.2.1 Estimating Confidence for Multiple Measurements

Consider a particular quantity that needs to be measured, such as temperature. Assume that we have two temperature sensors that measure the temperature of the target. The question for this part is, using the two measurements, what is the best estimate of temperature that can be obtained?

Let the two temperature measurements be mathematically expressed as t_1 and t_2 . We can simply estimate the temperature by taking the mean $\hat{t} = 0.5t_1 + 0.5t_2$, but it

may not be the best estimate. Assuming that the best estimate is given by the following expression, we need to choose the best possible α .

$$\hat{t} = (1 - \alpha) t_1 + \alpha t_2$$

Intuitively, α will be a measure of the relative confidence that we have in the sensor that takes the measurement t_2 . Mathematically, a good approximation of this confidence will be inversely proportional to the variance of the measurements t_2 . If the measurements of a particular sensor have very high variance, they are less precise and hence it would represent relatively less confidence on the sensor. Our aim is to improve the confidence of $\hat{t}$. If the measurements t_1 and t_2 have the variances σ_1 and σ_2 , then the variance of $\hat{t}$ would be as follows [4].

$$\sigma_{\hat{t}}^2 = (1 - \alpha)^2 \sigma_1^2 + \alpha^2 \sigma_2^2$$

The maximum confidence or minimum variance of $\hat{t}$ would correspond to a zero derivative of the above expression with respect to α . The optimum value obtained for α is called the Kalman gain, K , and is given by Equation 4.3.

$$K = \frac{1/\sigma_1^2}{(1/\sigma_1^2) + (1/\sigma_2^2)} \quad (4.3)$$

Using the obtained value of K , the best estimate of temperature and the variance of the estimate can be expressed as Equations 4.4 and 4.5.

$$\hat{t} = t_1 + K(t_2 - t_1) \quad (4.4)$$

$$\sigma_{\hat{t}}^2 = (1 - K)\sigma_1^2 \quad (4.5)$$

Here the temperature we are estimating is a scalar. However, the same would be applicable for vectors, with variance replaced by covariance matrix. In this case, Equations 4.3, 4.4 and 4.5 will become Equations 4.6, 4.7 and 4.8 respectively, where Σ_1 and Σ_2 are the covariance matrices of measurements $\vec{x}_1$ and $\vec{x}_2$.

$$K = \Sigma_1(\Sigma_1 + \Sigma_2)^{-1} \quad (4.6)$$

$$\hat{\vec{x}} = \vec{x}_1 + K(\vec{x}_2 - \vec{x}_1) \quad (4.7)$$

$$\Sigma = (I - K)\Sigma_1 \quad (4.8)$$

This idea of obtaining the best possible estimate from multiple measurements based on their covariance is a key idea used in Kalman filtering.

4.2.2 Prediction Correction Method

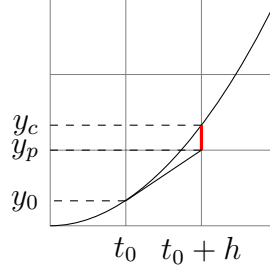


Figure 4.3: Prediction Correction Method

The Prediction-Correction method is a numerical method used to solve ordinary differential equations. As the name suggests, it has two steps. The first step involves prediction of the value and the second step involves correction. Consider a differential equation.

$$\frac{dy(t)}{dt} = f(y, t)$$

If the initial value is known to be $y(t_0) = y_0$, then we could predict the subsequent value using a numerical technique like the Euler Forward Method.

$$y_p = y_0 + hf(y_0, t_0)$$

Since the predicted value differs from the actual value, the next step corrects the value by performing iterations until the error $(y_c^{(n+1)} - y_c^{(n)})$ converges below a threshold. The initial value of y_c^k is assumed to be the predicted value y_p , and the final value is the corrected value y_c .

$$y_c^{(n+1)} = y_0 + h \frac{f(y_0, t_0) + f(y_c^{(n)}, t_0 + h)}{2}$$

In the above expression, instead of considering the slope at t_0 , iterations are performed for the mean slope at t_0 and $t_0 + h$. This gets the corrected value closer to the actual value.

4.2.3 Kalman Filtering

Kalman filtering is a filtering technique that combines the two previously mentioned methods. It predicts the state of the system using a user-defined model of the system.

A measurement of the system is also obtained by using suitable sensors, which is the LiDAR in this case. The predicted and measured states are then used to obtain the estimated state on the basis of the covariance [5].

The prediction step predicts the state of the system. Let $\vec{x}$ be the state of the system that is governed by the following equation, where $\vec{u}$ is the input given to the system. The p and c in the subscript stand for *predicted* and *corrected*.

$$\vec{x}_p^{(n)} = A\vec{x}_c^{(n-1)} + B\vec{u}^{(n)} \quad (4.9)$$

The corresponding covariance for the predicted state will be given by the following equation, where Q is the covariance of the control error.

$$\Sigma_p^{(n)} = A\Sigma_c^{(n-1)}A^T + Q \quad (4.10)$$

The correction step then involves combining the corrected state with a measured state. Let $\vec{x}_m^{(k)}$ denote the measured state and R denote the covariance of the measurement error. According to Equation 4.6, the Kalman gain will be given by the following equation.

$$K^{(n)} = \Sigma_p^{(n)}(\Sigma_p^{(n)} + R)^{-1} \quad (4.11)$$

Using the obtained Kalman gain, the predicted values for state and covariance can be corrected according to Equations 4.7 and 4.8.

$$\vec{x}_c^{(n)} = \vec{x}_p^{(n)} + K \left(\vec{x}_m^{(n)} - \vec{x}_c^{(n)} \right) \quad (4.12)$$

$$\Sigma_c^{(n)} = (I - K)\Sigma_p^{(n)} \quad (4.13)$$

However, in some cases, the sensor might not measure the entire state, but just measure some parameters of the state. In such cases, a transformation matrix is required to convert the measured data to the state. In this case, let $\vec{z}^{(n)}$ be the measured data, where $z^{(n)}$ is governed by the following equation.

$$z^{(n)} = H\vec{x}^{(n)} \quad (4.14)$$

This needs to be accommodated in Equations 4.11, 4.12 and 4.13. The Kalman filtering algorithm with these accommodations is shown in Figure 4.4

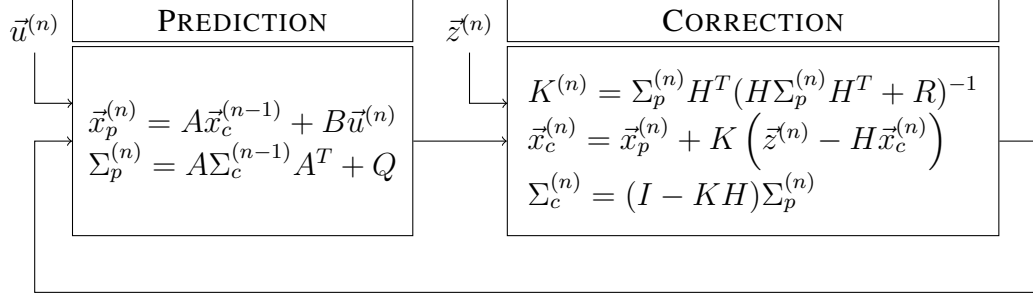


Figure 4.4: Kalman Filter

4.3 Filtering Orientation Estimates

Like the position, the orientation data needs to be filtered to eliminate any form of noise. The relative orientation, measured as a quaternion by the ICP algorithm, is converted to the global frame and passed through a low-pass filter to obtain the estimated orientation. Subsequently, the available orientation data can be numerically differentiated to derive the estimated angular velocity, $\vec{\omega}_e$.

4.3.1 Low Pass Filter Design

The low-pass filter is implemented to primarily control the noise. A conventional first-order low pass has the following transfer function, where ω_c is the cutoff frequency.

$$H(s) = \frac{K}{1 + s/\omega_c} \quad (4.15)$$

where $s = \sigma + j\Omega$ is the complex frequency, and K is the gain. To analyze the filter's behavior in the frequency domain, we substitute $s = j\omega$, yielding the following frequency response.

$$H(j\omega) = \frac{K\omega_c}{\omega_c + j\omega}$$

The magnitude and phase of this frequency response are given by:

$$|H(j\omega)| = \frac{\omega_c}{\sqrt{\omega_c^2 + \omega^2}}$$

$$\angle H(j\omega) = -\arctan\left(\frac{\omega}{\omega_c}\right)$$

From these equations it is observed that the magnitude response unity in the pass-band, where the frequency is much lower than the cutoff frequency ω_c . At the cutoff frequency ω_c , the magnitude drops to $\frac{1}{\sqrt{2}}$. In the stopband where the frequency is signif-

icantly higher than the cutoff frequency, the magnitude of the filter output decreases by a factor of 10 each time the frequency increases by a factor of 10. The phase response starts at 0° in the passband, shifts to -45° at the cutoff, and asymptotically approaches -90° in the stopband. The Bode plots for the low-pass filter are shown in Figure 4.5, where the red line shows the cutoff frequency. The gain is expressed as decibel (dB). A magnitude of $20 \log(A)$ dB corresponds to a linear gain of A .

Having a very high cutoff frequency would pass some noisy frequencies, while having a very low cutoff frequency would introduce significant phase shift in the lower frequencies.

The transfer function is converted to the discrete domain by using the bilinear transformation, which expresses the complex variable s as a function of z . The bilinear transform function actually maps the complex s -plane and the complex z -plane, where the imaginary axis of the s -plane becomes the unit circle in the z -plane.

$$s = \frac{2}{t_0} \left(\frac{1 - z^{-1}}{1 + z^{-1}} \right)$$

In the above expression, t_0 is the sampling period. Substituting the bilinear transformation in Equation 4.15 gives the following equation:

$$H(z) = \frac{\omega_c t_0 (1 + z^{-1})}{2(1 - z^{-1}) + \omega_c t_0 (1 + z^{-1})}$$

which, on conversion to the time domain, translates to the following equation

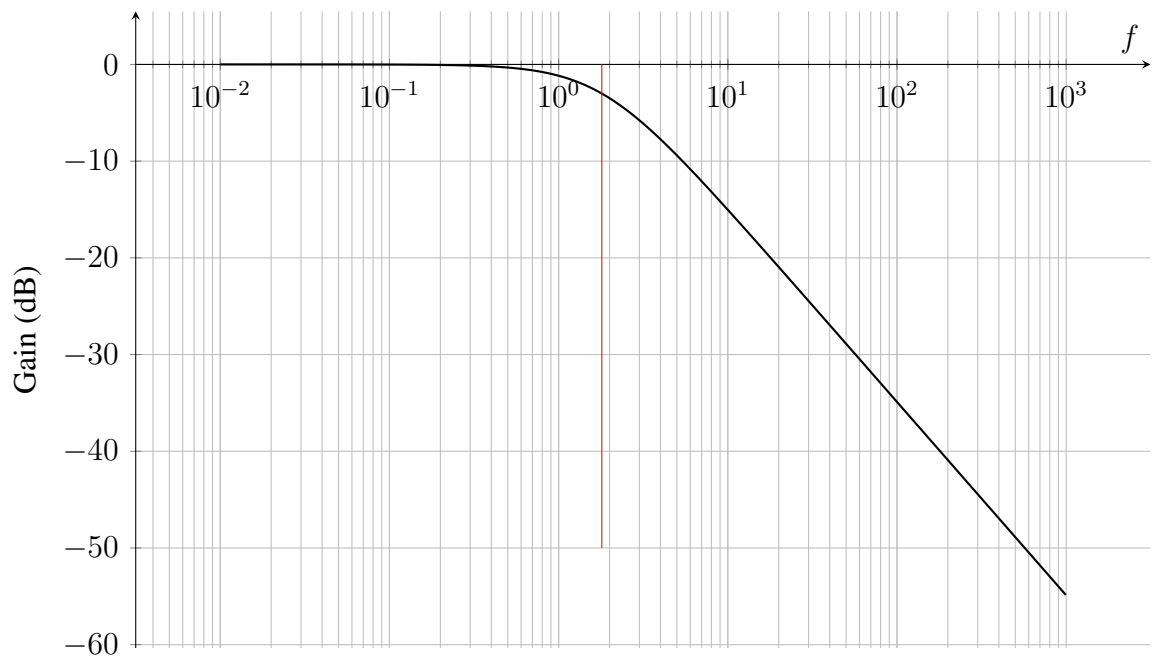
$$y[n] = \frac{\omega_c t_0 (x[n] + x[n-1]) + (2 - \omega_c t_0) y[n-1]}{2 + \omega_c t_0}$$

4.4 Pose Estimation Algorithm

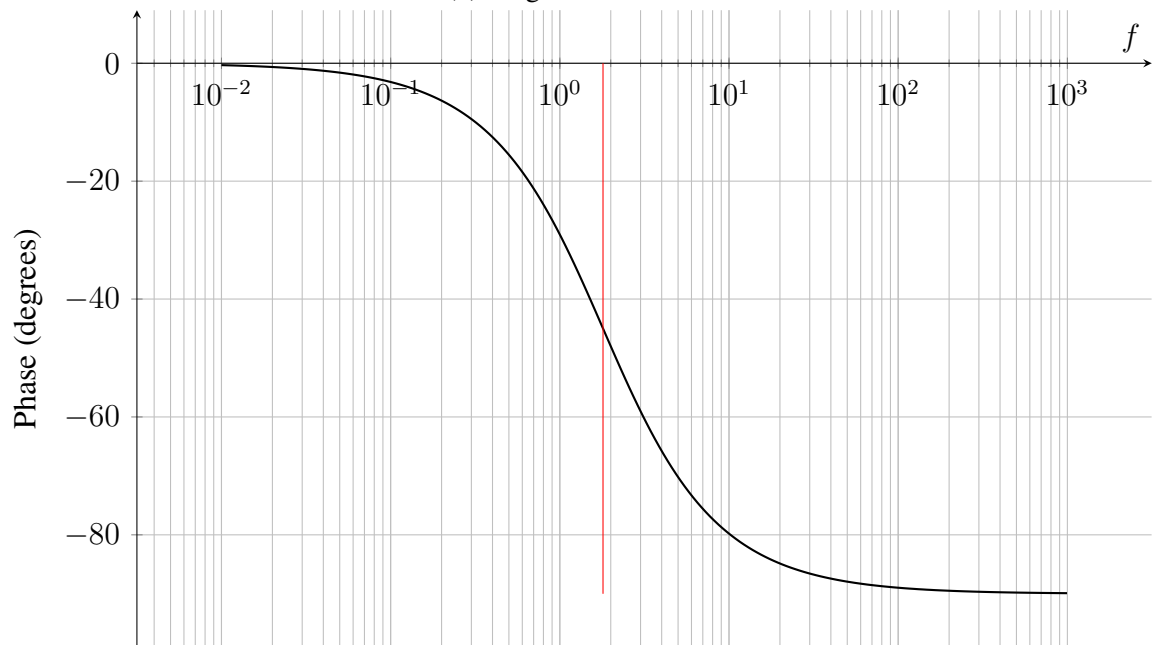
Having covered the necessary tools for pose estimation, such as the ICP algorithm and Kalman filtering, this section focuses on applying these methods to estimate the pose. The states estimated as part of the pose estimation algorithm are position, orientation, linear, and angular velocities.

Assuming the target to be a rigid body, these states can be put into context by estimating position and velocity for the point of interest of the target, and estimating the orientation and the angular velocity about the point of interest.

The LiDAR captures images at a specific frequency and, for each image, there is a corresponding reference. As shown in Figure 4.6, the pose estimation algorithm is illustrated, where m and e in the subscripts denote ‘measured’ and ‘estimated’, respectively. Initially, both the reference image and the captured LiDAR image are inputs to



(a) Magnitude Plot



(b) Phase Plot

Figure 4.5: Bode Plot for Low Pass Filter

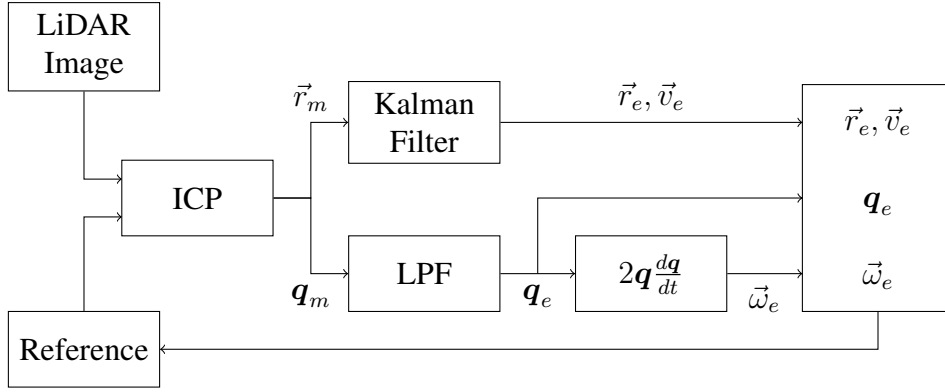


Figure 4.6: Pose Estimation Algorithm

the ICP algorithm, which computes the relative position $\vec{r}_m$ and the relative orientation $\mathbf{q}_m$. The relative position is then transformed into the global coordinate frame and provided as a measured state to the Kalman filter, which estimates the position and velocity, as discussed in the previous chapter. Similarly, the orientation data is filtered using the low-pass filter and processed to get the angular velocity. With all four states estimated, they are used to construct a new reference by applying rotation and translation to the previous reference based on the estimated states. This updated reference is then used as the starting point for the next image in the ICP algorithm.

4.5 State-based Sensor Workflow

Having established the core estimation algorithm, the next step involves designing the sensor architecture. The target may not always be within the FoV when the sensor is activated, and it may enter or exit the FoV dynamically during operation. Such scenarios can lead to inaccurate estimation results or cause the algorithm to fail. To address these challenges, a state-based approach is proposed to govern the operation and ensure robust estimation under varying visibility conditions. Each state represents a specific operational mode determined by the visibility and stability of the target within the LiDAR's field of view.

1. **Blind Zone:** This state is entered when the flash LiDAR captures an empty image, indicating the absence of any detectable target within its field of view (FoV). Upon entering the Blind Zone, a search algorithm is initiated, prompting the LiDAR system to rotate or reposition, in order to systematically scan the surrounding area. This search continues until a non-empty image is acquired, at which point the system transitions to the Partial Image state.
2. **Partial Image:** This state is characterized by the continuous capture of non-empty

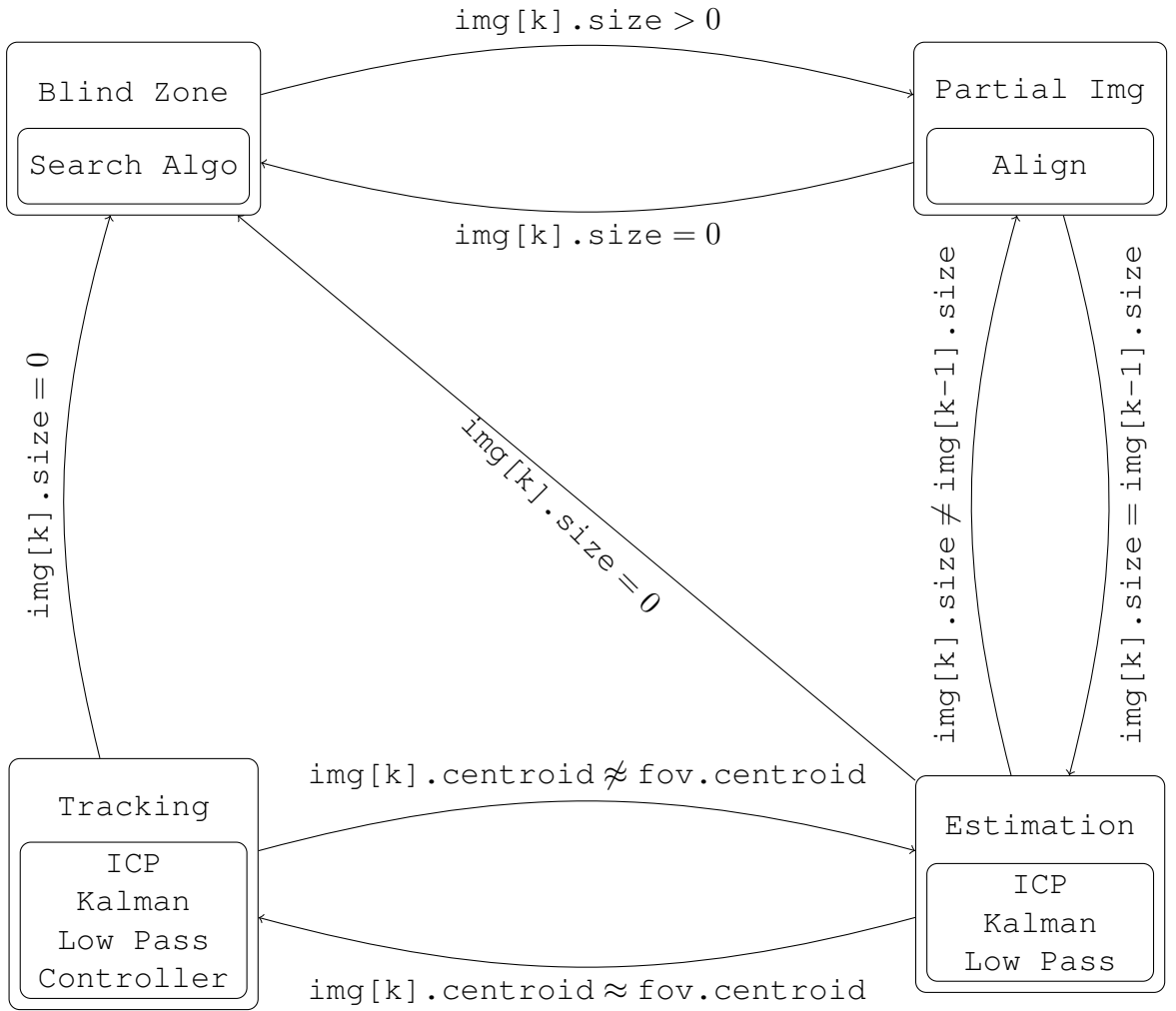


Figure 4.7: Sensor Workflow

images, where the number of detected points on the target fluctuates significantly. During this phase, the chaser attempts to align itself such that the centroid of the target aligns closely with the centroid of the LiDAR's field of view. This alignment facilitates the visibility of the maximum possible area of the target. Once the number of points stabilizes across consecutive frames, the system transitions to the Estimation state.

3. **Estimation:** In this state, the complete pose estimation algorithm (Figure 4.6) is executed to determine the target's pose and velocities. The kinematic parameters of the target are identified based on the LiDAR data. If the centroid of the target remains closely aligned with the centroid of the field of view and the number of detected points remains stable, the chaser can move to the next state.
4. **Tracking:** The tracking state is an extension of the estimation state. In addition to

all the algorithms running in the estimation state, the estimated pose and velocity data are fed into a control system that tracks the target. The system remains in the ‘Tracking’ state as long as the target remains within the field of view and the number of detected points is stable. If significant fluctuations occur or the target exits the field of view, the chaser transitions back to the Estimation state.

This structured state flow enables the chaser to switch between search, alignment, estimation, and control behaviors based on sensor feedback. Using the estimated pose with real-time control logic, the system can autonomously find and track the target, even in the presence of environmental uncertainty or partial observations.

4.5.1 Estimation without Reference

All the estimation performed so far has been heavily dependent on the reference image stored in the memory of the chaser. This is because the ICP algorithm, which acts as the basis of our estimation, needs a reference image to compare with a target and estimate the states.

If the reference is unavailable or produces inaccurate results, then the algorithm should be able to estimate states without the reference. To do this, consecutive images are used as reference and target. This would give the difference in position and orientation between consecutive frames. In such cases, the position and orientation estimates would be relative and would need some calibration and transformation by the chaser to make sense of the data. However, the estimated velocities would still be valid and could be used to track the target.

The major drawback with this case is that the measured values will have significantly greater noise, because, the reference and the target, both are noisy. The previously designed filters, specifically the low-pass filter does not provide accurate results, and would work only for a different cutoff frequency.

4.5.2 Implementation Scenario

The sensor workflow was implemented for a one-dimensional case, considering rotation only about the z axis. A robust workflow for three dimensions can be developed with the same states and a similar logic flow. However, the key difference is that in one dimension, aligning does not require a separate algorithm. The search algorithm itself gets the chaser to the aligned condition (where centroids of the target and FoV are close), after which estimation starts. However, in three-dimensional case, aligning would need a separate control sequence.

Figure 4.8 shows the main states and transitions in the implementation of the sensor workflow. The following sequence outlines the simulation workflow designed to emulate the behavior of a chaser system for pose estimation in a target-chaser scenario with rotation about single axis:

- (a) Initial Case: The simulation begins with the chaser oriented such that the target lies along the negative y axis, effectively placing it outside the chaser's initial field of view (FoV). Consequently, the Flash LiDAR system captures empty frames, resulting in a 'blind mode' operation. This lack of target detection initiates the activation of the search algorithm.
- (b) Search Algorithm: In the absence of target detection, the chaser engages a search protocol that involves rotation about the z axis. This rotational motion is governed by a closed-loop control mechanism, which computes the required torque to achieve and maintain a desired angular velocity. The chaser continues this rotational motion until the target is detected within the LiDAR's FoV.
- (c) Partial Image (Entering): As the chaser's FoV begins to intersect with the target, the LiDAR system captures an increasing number of points in successive frames. This phase is classified as "Partial Image Mode." Due to the unstable and fluctuating nature of the point cloud in this phase, the chaser remains in this mode without transitioning to estimation until a stable point cloud is observed.
- (d) Estimation: Once the target is fully within the FoV, the point cloud data stabilizes, indicating a consistent target presence. At this stage, pose estimation begins. Since the reference frame data is unavailable, the estimation process solely relies on velocity measurements to determine the relative motion between the chaser and the target.
- (e) Tracking: With the target entirely visible and its centroid aligned with the center of the chaser's FoV, the system transitions to tracking mode. In this implementation, tracking is realized through a co-rotational mechanism. This involves a proportional controller that utilizes inputs from the estimation module to maintain alignment with the target's motion.
- (f) Partial Image (Exiting): To simulate the target exiting the FoV, the chaser is deliberately subjected to an external torque, overriding the control action and inducing rotation. As the target moves out of view, the number of detected LiDAR points decreases, leading the system to re-enter Partial Image Mode. Eventually, with the target no longer visible, the chaser reverts to blind mode, resuming its search-like rotational behavior.

- (g) Continuation: Upon completing a full 360-degree rotation, the chaser's FoV once again intersects with the target. This triggers a repetition of the aforementioned sequence, thereby simulating a continuous target detection, estimation, and tracking cycle.

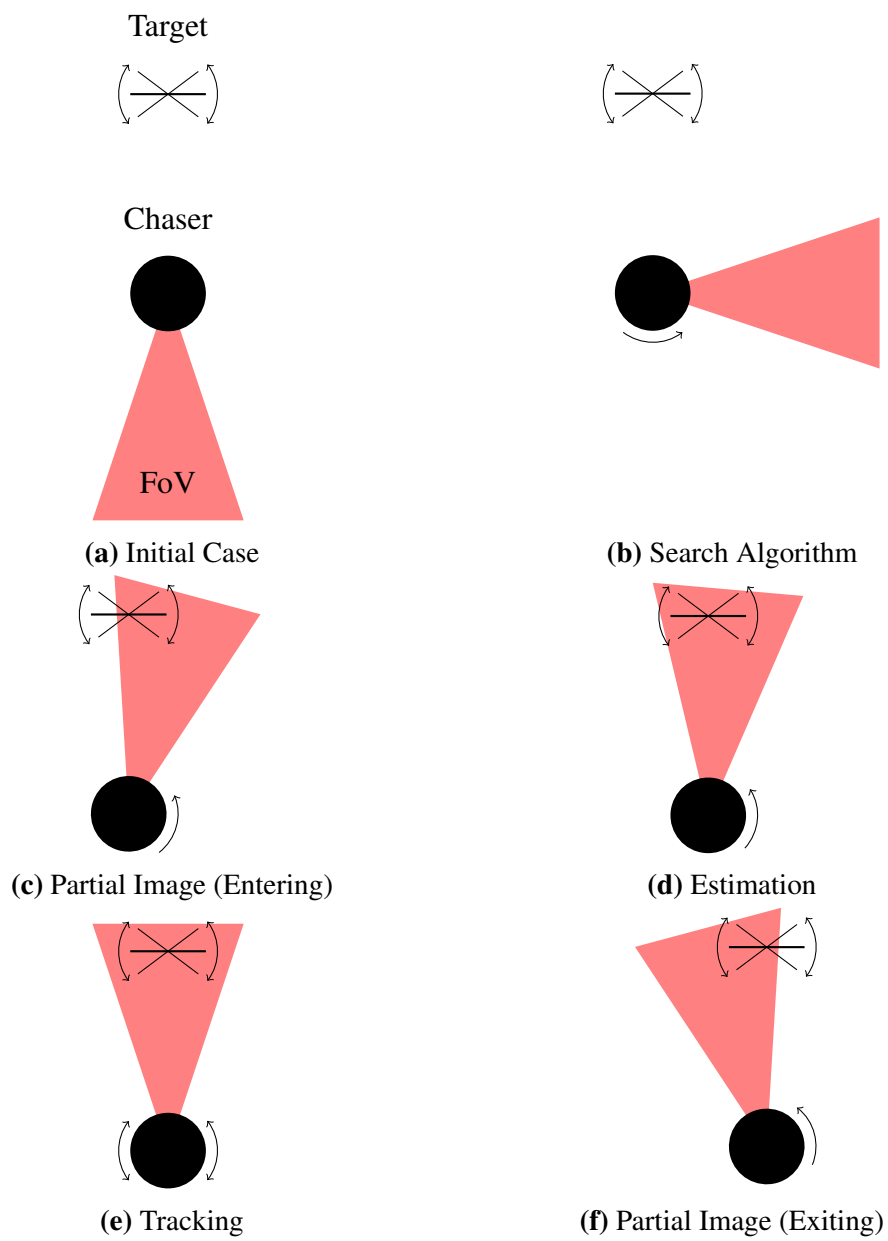


Figure 4.8: Sensor Workflow

Chapter 5

Implementation and Results

The entire pose estimation algorithm was implemented in C++, using a self-written matrix and pose library.

5.1 Rigid Body Simulations

To understand the rigid body kinematics and dynamics and to test the matrix library for an arbitrary rigid body, the rotation of a rigid body was simulated for 20000 seconds using the Runge-Kutta 4 method.

Equation 3.5 is a differential equation in $\vec{\omega}$ that can be solved using standard iterative techniques such as the Runge-Kutta Order 4 (RK4) method. The RK4 method is based on the approximation of the solution to a differential equation at each step using four increments. The general form of the RK4 method uses the initial value $y(t_0) = y_0$ and solves the differential equation $\frac{dy}{dt} = f(t, y)$ using

$$y_{n+1} = y_n + \frac{1}{6} (k_1 + 2k_2 + 2k_3 + k_4)$$

where

$$\begin{aligned} k_1 &= h \cdot f(t_n, y_n) \\ k_2 &= h \cdot f\left(t_n + \frac{h}{2}, y_n + \frac{k_1}{2}\right) \\ k_3 &= h \cdot f\left(t_n + \frac{h}{2}, y_n + \frac{k_2}{2}\right) \\ k_4 &= h \cdot f(t_n + h, y_n + k_3) \end{aligned}$$

and h is the step size (the difference between successive values of t). $\vec{\omega}$ was calculated for a period of 20000 seconds assuming the following values of the initial $\vec{\omega}_0, I$ and $h = 0.01$ seconds.

$$I = \begin{bmatrix} 25 & -0.5 & 1.5 \\ -0.5 & 30 & 2 \\ 1.5 & 2 & 28 \end{bmatrix}, \quad \vec{\omega} = (\hat{i} - \hat{j} + \hat{k})^\circ/s$$

Figure 5.1 shows the components of $\vec{\omega}$ vary over 20000 seconds. The angular momentum $\vec{h}$ remained constant throughout the simulation time while no torque was applied. The constant angular momentum verified the simulation results were correct.

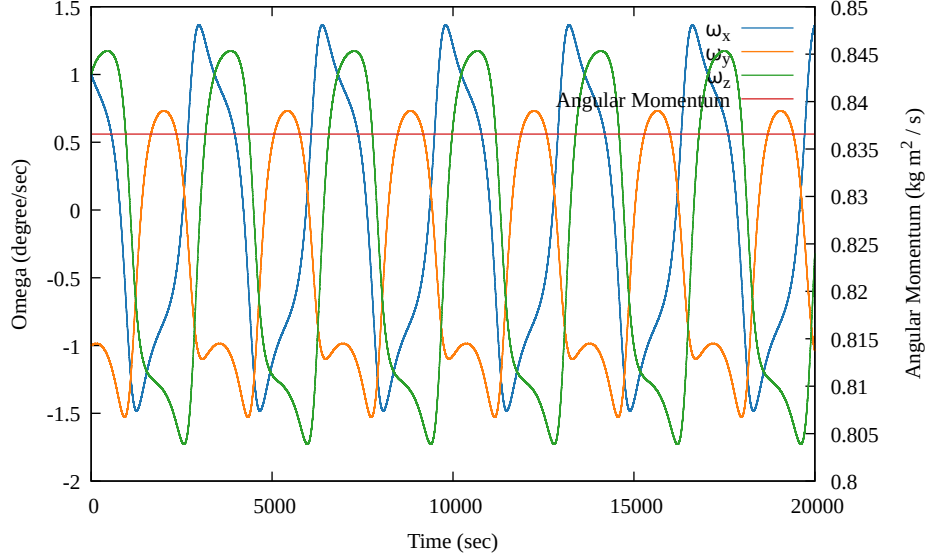


Figure 5.1: Components of $\vec{\omega}$ for 20000 seconds

The obtained $\vec{\omega}$ was again integrated using Equation 3.2 with the RK4 method to obtain the corresponding quaternions, which were then converted to Euler angles for easier interpretation. Figure 5.2 shows the variation of the angles ϕ , θ and ψ during the 20000 seconds and for the first 4000 seconds for better clarity. The graphs for Euler angles might look discontinuous, but that is due to the nature of Euler angles and not due to discontinuous orientation. There could be multiple Euler angle possibilities to express the same orientation. For consistency, ϕ and ψ are assumed to be between -180° and 180° , and θ is assumed to be between 0° and 90° . Taking this into account, the continuity of orientation can be verified.

5.2 Reference and Target Images for ICP

The reference images were obtained from the Laboratory for Electro-Optics Systems (LEOS). As shown in Figure 5.4, the images were taken using the Ouster OS2 Flash LiDAR [6] in a laboratory with a model of the SPADEX satellite as the target. The image also involved an unwanted background other than the target, and the target was

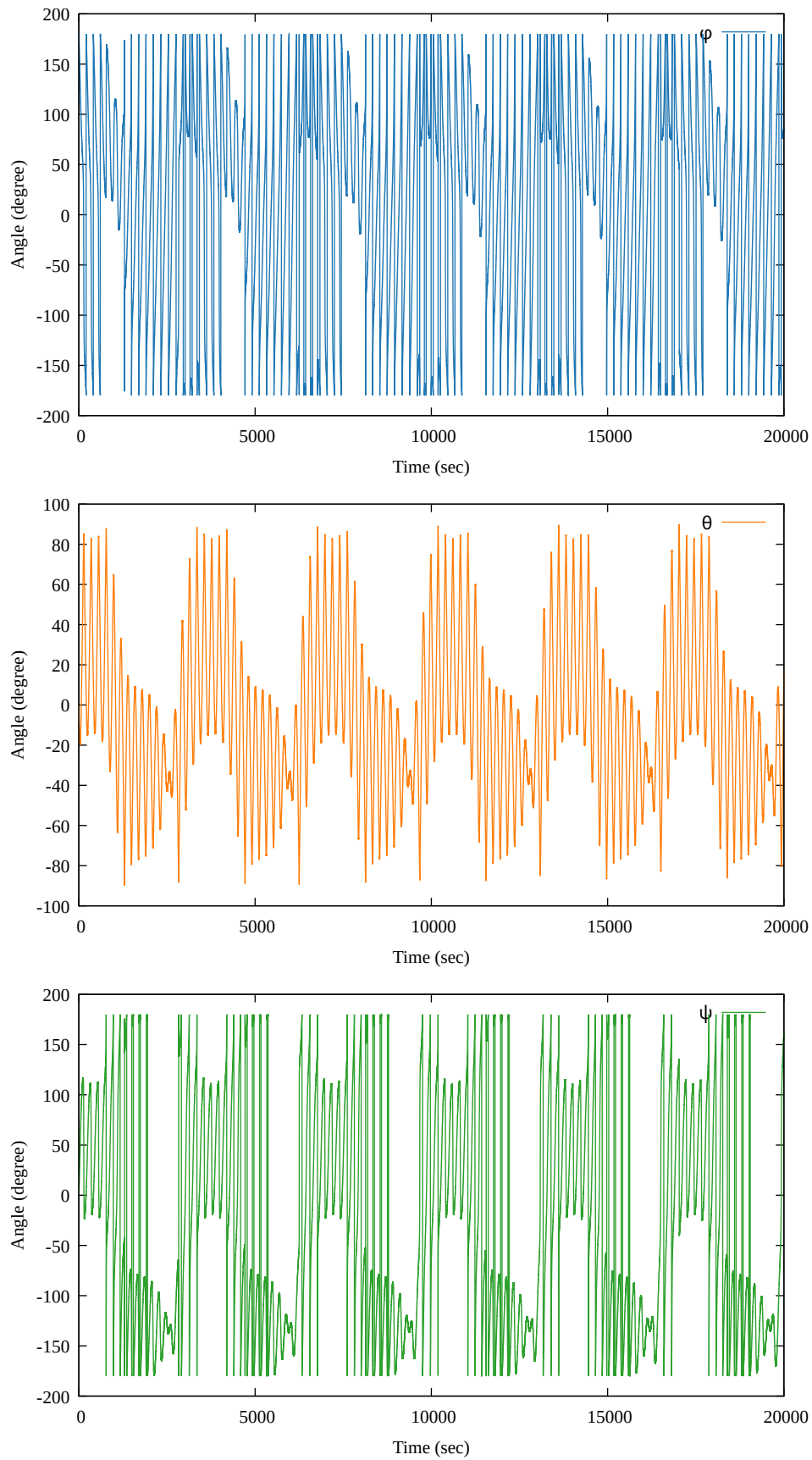


Figure 5.2: Euler Angles for 20000 seconds

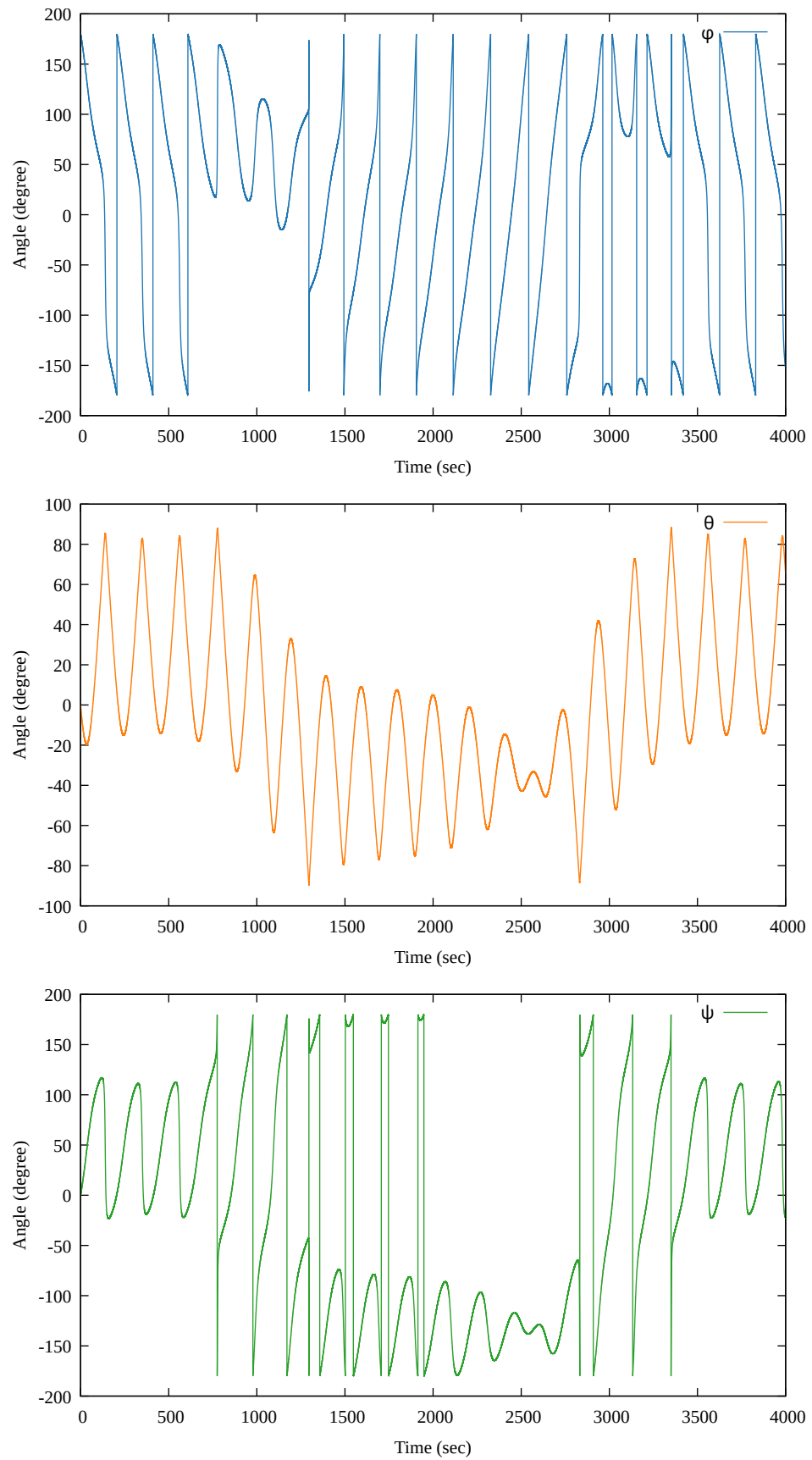


Figure 5.3: Euler Angles for 4000 seconds

extracted using the clustering algorithm developed during the internship. The target images were generated by adding some white Gaussian noise with standard deviation 5% of the average distance between the points to the coordinates of the points in the point cloud.

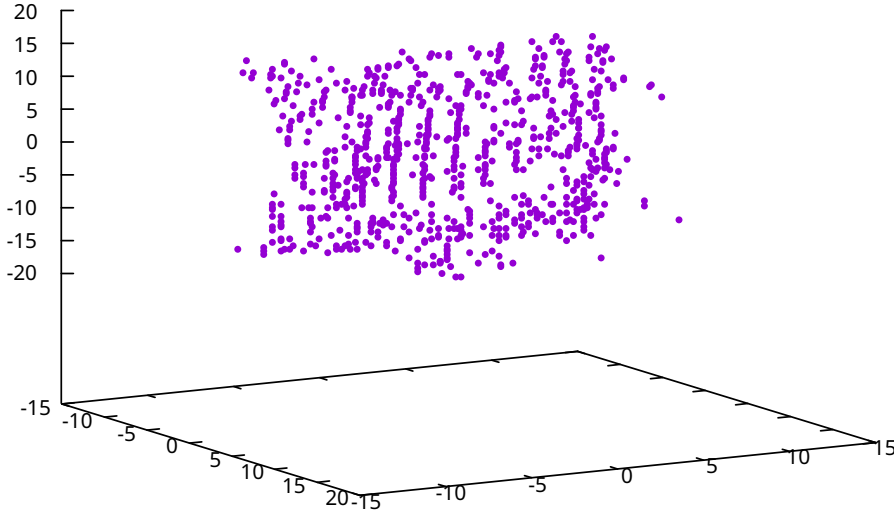


Figure 5.4: Reference LiDAR Image

5.3 Kalman Filter Implementation

In the context of pose estimation, the Kalman filter is mainly used to estimate position and velocity. Therefore, the states in the Kalman filter are position, $\vec{r}$, and velocity, $\vec{v}$. However, since position can be measured from the LiDAR image, the observed state, $\vec{z}$, is limited to position. The input $\vec{u}$ represents the applied force and the step size is denoted by Δt . The matrices A , B , and H can be obtained from the following equations.

$$\begin{bmatrix} \vec{r} \\ \vec{v} \end{bmatrix} = \begin{bmatrix} I & hI \\ 0 & I \end{bmatrix} \begin{bmatrix} \vec{r} \\ \vec{v} \end{bmatrix} + \begin{bmatrix} h^2/2 \\ h \end{bmatrix} \frac{\vec{F}_{ext}}{m}$$

$$\vec{z} = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} \vec{r} \\ \vec{v} \end{bmatrix}$$

The step size, $h = 0.1$, and the filter is applied to the point of interest of the target. For the code implementation of the Kalman filter, the components of position and velocity are handled separately as shown in Appendix A.5. Hence, I in the above equation refers to the identity matrix, which makes A a 6×6 matrix. Similarly, the matrices B and H are extended in size to fit the dimensions. The control error, Q , was neglected for simulation purposes and can be assumed based on the data in [6]. The measurement error, R , was considered based on the noise profile of the measured data.

The Kalman filter was applied to estimate purely translational motion, with a 5% measurement noise introduced to the point cloud. Figures 5.5 and 5.6 presents the results, displaying the comparison between the measured and estimated data. Additionally, it shows the square root of the trace of the covariance matrix, which varies with each iteration, reflecting the evolving uncertainty in the estimates over time, demonstrating the filter's effectiveness in reducing estimation error.

5.4 Low Pass Filter Results

The first order discrete low pass filter was designed as mentioned in the previous section. As the frequency of noise is not fixed, an arbitrary cut-off frequency of the order of $0.1/T$. The filtered values of q and $\vec{\omega}$ are shown in Figures 5.7 and 5.8. The filter was applied to the orientation (quaternion) data given by the algorithm, and the velocity, $\vec{\omega}$ was calculated using filtered orientation data.

5.5 Sensor Workflow Implementation

The sensor workflow was implemented for a preliminary case, where rotation about only one axis is involved. The target is behind the FoV of the chaser, and is oscillating about the z axis. The search algorithm for the chaser is to rotate with $\vec{\omega} = 1^\circ/s\hat{k}$, which is a simplified case, as the search algorithm is implemented for single dimension.

The results corresponding to the simulated chaser-target scenario are presented in this section. They reflect the sequential behavior of the chaser system under the Flash LiDAR-based pose estimation framework. The simulation captures the transition from blind mode to search, partial detection, estimation, tracking, and back to partial and blind modes as the target moves in and out of the field of view. The observed point cloud dynamics, rotational control responses, and estimation outputs are presented in Figure 5.9. In the figure, the gray background represents blind zone, orange background represents partial image, pink background represents estimation state and blue background represents.

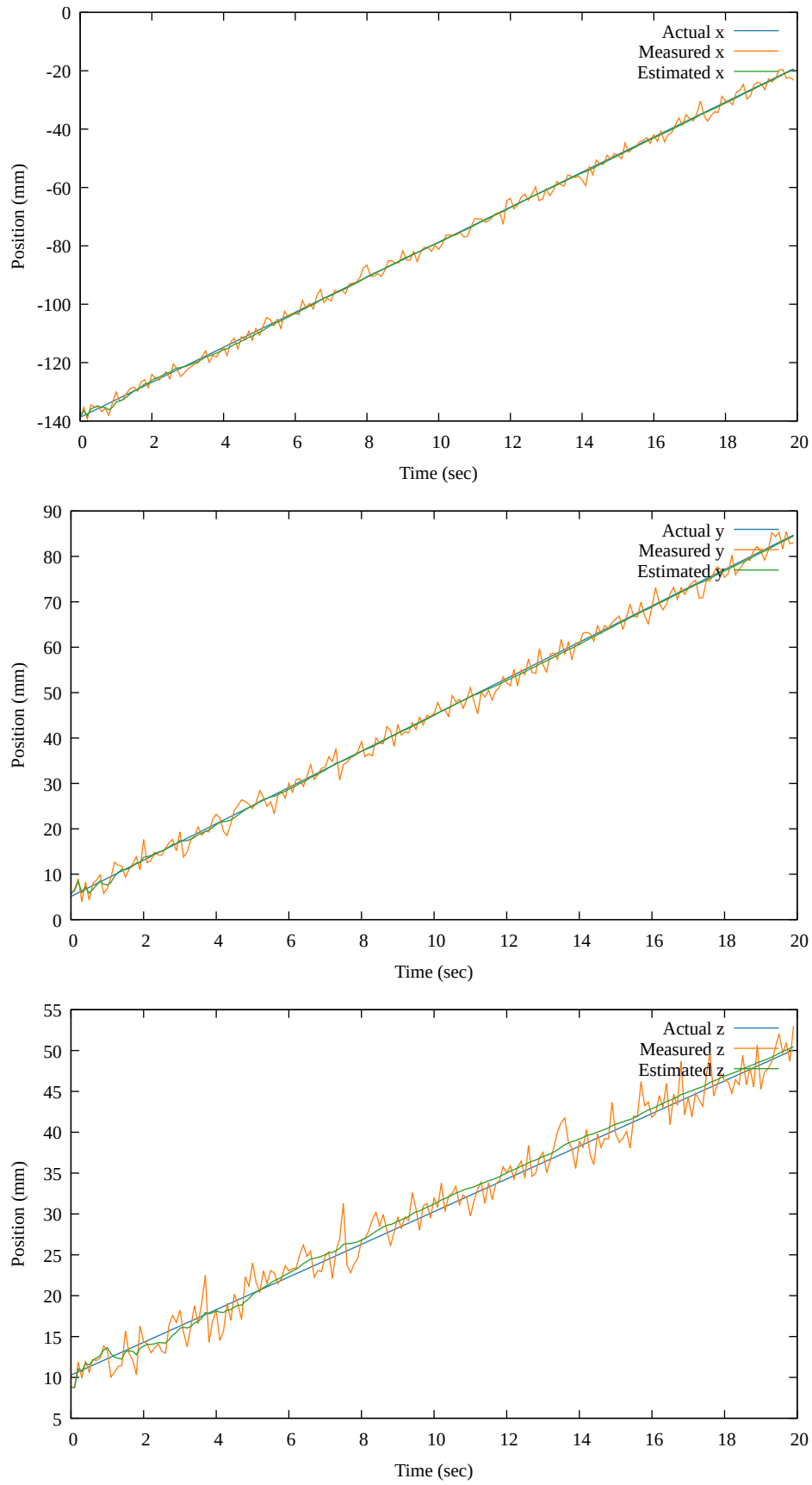


Figure 5.5: Kalman Position Results

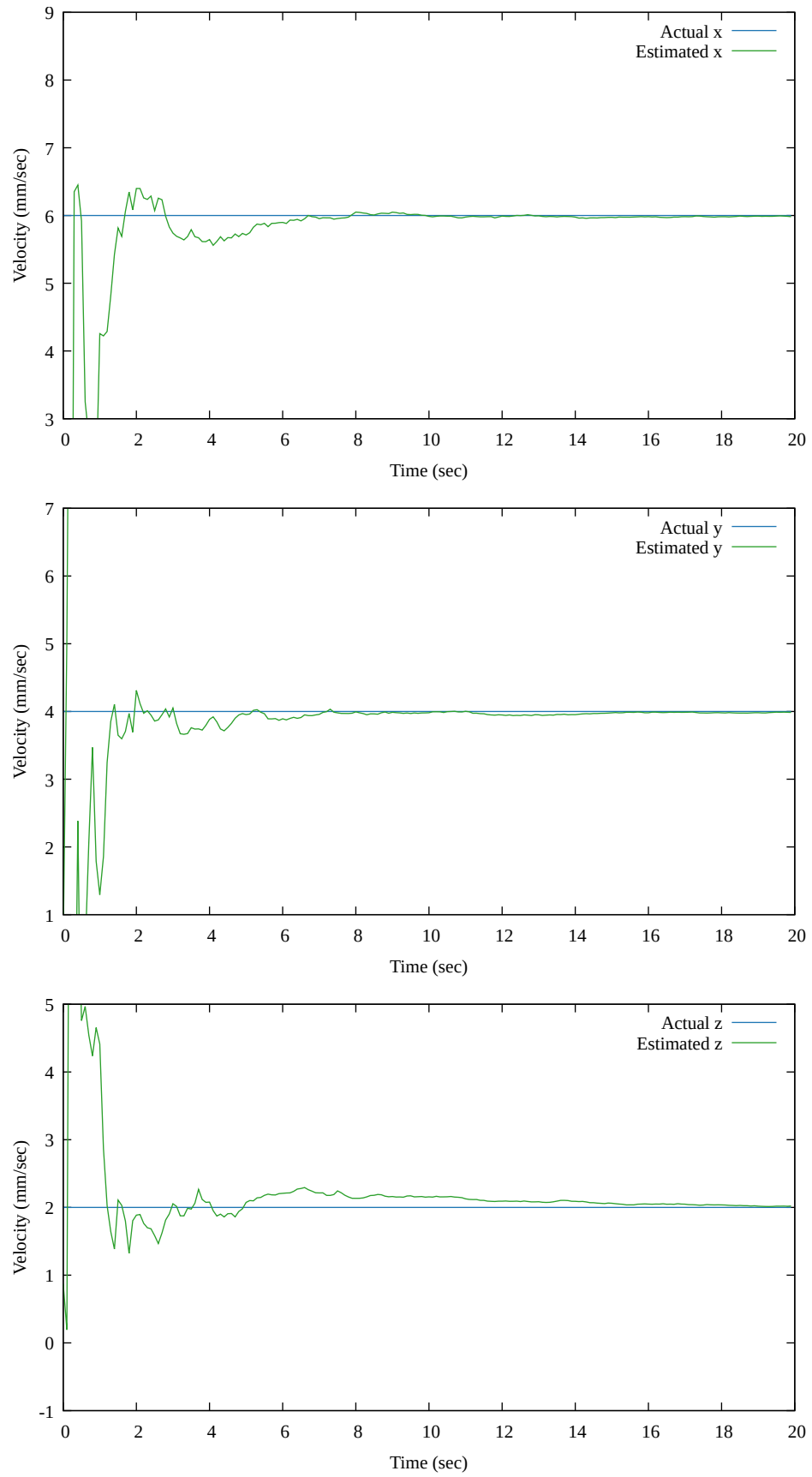


Figure 5.6: Kalman Velocity Results

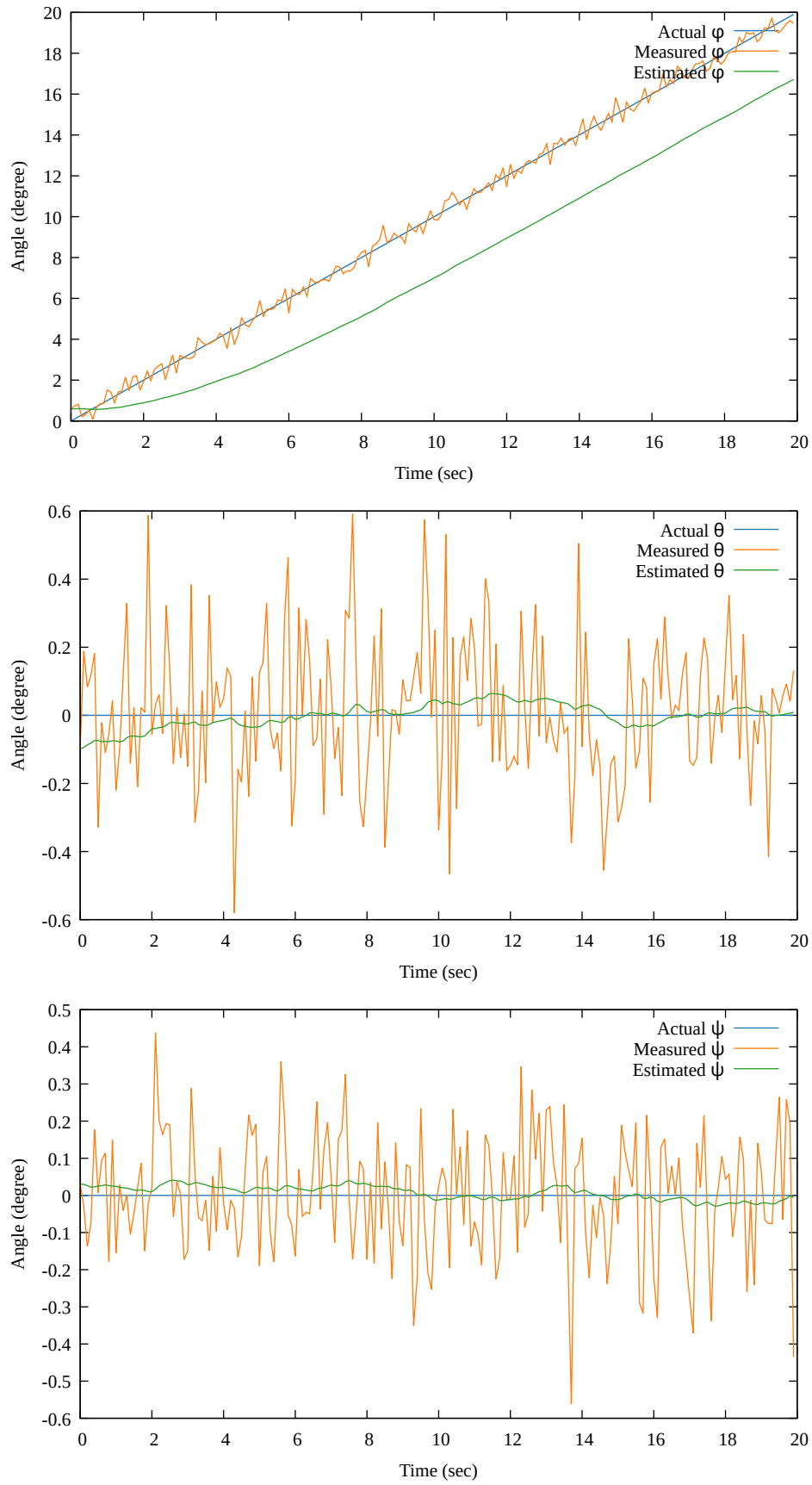


Figure 5.7: Low Pass Filter Angles Results

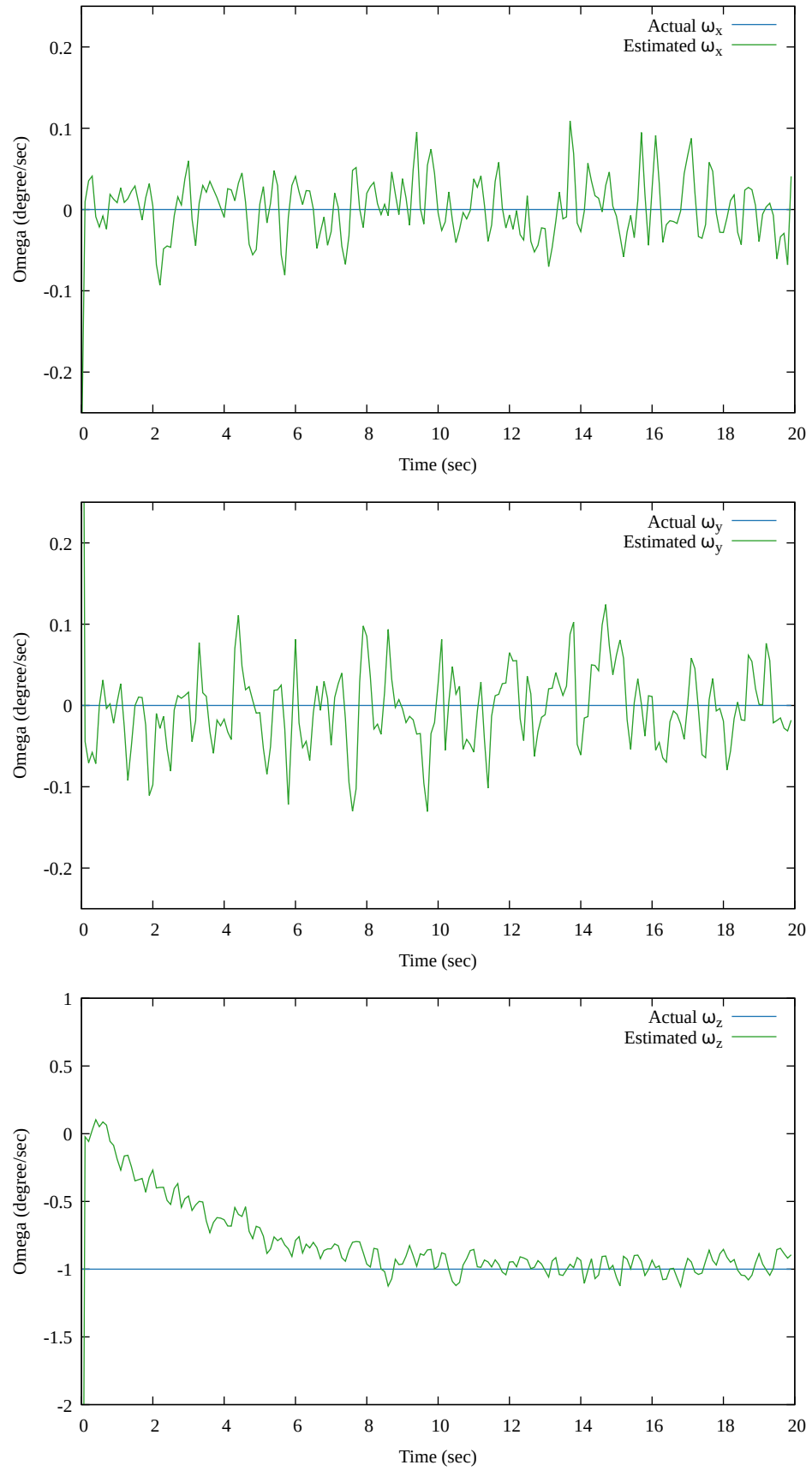


Figure 5.8: Low Pass Filter Velocity Results

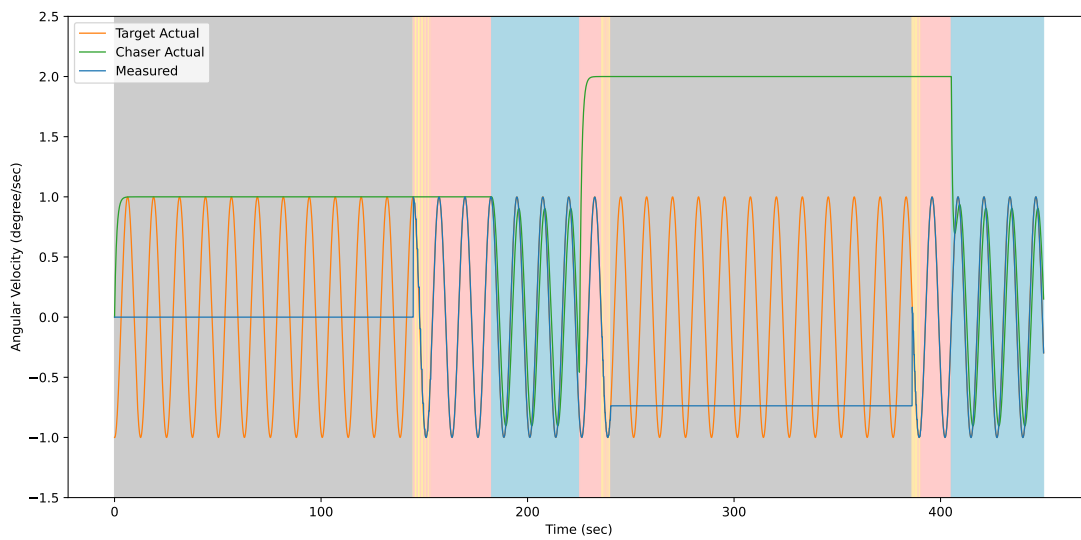


Figure 5.9: Sensor Workflow Results

Chapter 6

Conclusion and Future Work

This project involved the development of a framework for pose estimation using Flash LiDAR, with the aim of estimating the position, orientation, and velocities of a target relative to a chaser. The initial phase of the project included a comprehensive review of rigid body kinematics and dynamics, which provided the foundation for understanding the motion of both the target and chaser. To support these models, C++ libraries were developed to implement the necessary kinematic and dynamic equations, and these libraries were thoroughly tested with a 20,000-second simulation, demonstrating their reliability and effectiveness in modeling the system's behavior.

The pose estimation algorithm was built around the ICP algorithm, which enabled optimal matching of point clouds captured by the Flash LiDAR sensor. Additionally, filtering techniques such as the Kalman Filter were incorporated to reduce noise in the data, significantly improving the accuracy and reliability of the pose estimations over time.

These methods were integrated into a sensor workflow that aimed to make the system more realistic and applicable to real-world scenarios. The chaser was able to autonomously detect, track, and lock onto the target in simulated environments. The results confirmed the robustness of the pose estimation system, with accurate tracking of the target's position and orientation. Moreover, the system successfully computed linear and angular velocities of the target, crucial for guiding the chaser's movements. Non ideal scenarios, such as the absence of a reference image, were also tested and handled, further demonstrating the system's adaptability and effectiveness.

6.1 Limitations

While the pose estimation framework developed in this project provides a solid foundation, there are several limitations that impact the system's performance in certain scenarios:

1. **Non-optimum Initial Viewing Angles:** The ICP algorithm relies heavily on the initial alignment of the point clouds for accurate matching. When the initial viewing angle of the target is non-optimal, the ICP algorithm struggles to converge to the correct solution, leading to poor pose estimation. This issue can arise particularly when the target and chaser are far apart or viewed from an edge-on angle.
2. **Noise Assumptions in Filtering Techniques:** The filtering algorithms like Kalman filter were designed with the assumption of additive white Gaussian noise in the sensor data. As a result, the system can't handle other types of noise. This limitation restricts the system's ability to perform optimally in environments with noise characteristics that deviate from this assumption.
3. **Single-Dimensional Sensor Workflow:** The sensor workflow was implemented and tested only in a single-dimensional case. There is scope to extend this to multiple dimensions.
4. **Absence of Physical Sensor Failures:** The current model does not account for potential physical failures or malfunctions of the Flash LiDAR sensor. The current algorithm is based on tweaking the data obtained from the LiDAR, and physical features need to be considered.

6.2 Future Possibilities

Firstly, the possible improvements include enhancements to address the limitations. First, the ICP algorithm could be improved to handle non-optimum initial viewing angles by incorporating feature-based as mentioned previously. Additionally, the sensor workflow could be expanded to support multidimensional cases by incorporating multi-dimensional point cloud processing, allowing for full 3D scanning, estimation and tracking.

Beyond improving these limitations, the framework could be extended to more complex scenarios, such as integrating multiple sensors on a single chaser to provide better data for pose estimation, or enabling cooperation between multiple chasers to track a single target.

Bibliography

- [1] D. Kleppner, *An Introduction To Mechanics*. Cambridge University Press, 2014.
- [2] B. K. Horn, “Closed-form solution of absolute orientation using unit quaternions,” *Josa a*, vol. 4, no. 4, pp. 629–642, 1987.
- [3] P. J. Besl and N. D. McKay, “Method for registration of 3-d shapes,” in *Sensor fusion IV: control paradigms and data structures*, vol. 1611. Spie, 1992, pp. 586–606.
- [4] Y. Pei, S. Biswas, D. S. Fussell, and K. Pingali, “An elementary introduction to kalman filtering,” *Communications of the ACM*, vol. 62, no. 11, pp. 122–133, 2019.
- [5] R. E. Kalman, “A new approach to linear filtering and prediction problems,” *Journal of Basic Engineering*, 1960.
- [6] *Ouster OS2 Long-Range High-Resolution Imaging Lidar*, Ouster, Inc., 2021, v2.5.x REV7. [Online]. Available: <https://data.ouster.io/downloads/datasheets/datasheet-rev7-v2p5-os2.pdf>

Appendix A

C++ Programs

A.1 Matrix Library

matrix.hpp

```
1  #ifndef MATRIX_HPP
2  #define MATRIX_HPP
3
4  #include <vector>
5
6  class matrix {
7      unsigned int r, c;
8      double* data = nullptr;
9  public:
10     // Constructor
11     matrix () : r(0), c(0), data(nullptr) {}
12     matrix (int x, int y) : r(x), c(y) {data = new double[x
        * y] ();}
13     matrix (const matrix&);
14     // Destructor
15     ~matrix ();
16     // Essential Functions
17     int rows() const {return r;}
18     int cols() const {return c;}
19     double& val (int, int);
20     double val (int, int) const;
21     // Operators
22     bool operator== (const matrix&) const;
23     matrix& operator= (const matrix&);
24     matrix operator+ (const matrix&) const;
```

```

25     matrix operator- (const matrix&) const;
26     matrix operator* (const double&) const;
27     matrix operator* (const matrix&) const;
28     // Other Functions
29     matrix transpose () const;
30     matrix gaussElim () const;
31     matrix inv () const;
32     matrix house() const;
33     matrix eigVal (double threshold) const;
34     matrix eigVec (double threshold) const;
35     // Validation Functions
36     bool isUpperTriangular (double threshold) const;
37     // Factorisation
38     std::vector<matrix> plu () const;
39     std::vector<matrix> qr () const;
40 };
41
42 matrix operator* (const double&, const matrix&);
43
44 matrix I (int);
45
46 matrix solnLinearEqns (const matrix&, const matrix&);
47
48 #endif

```

matrix.cpp

```

1  #include "matrix.hpp"
2  #include <iostream>
3  #include <stdexcept>
4  #include <cmath>
5  #include <vector>
6
7  matrix::matrix (const matrix& m) : r(m.rows()) , c(m.cols()) {
8      data = new double[r * c] ();
9      for (int i = 0; i < (r * c); i++) {
10         data[i] = m.data[i];
11     }
12 }
13
14 matrix::~matrix () {

```

```

15         if (data != nullptr) {
16             delete[] data;
17             data = nullptr;
18         }
19     }
20
21     double& matrix::val (int x, int y) {
22         if (x <= 0 || x > r) {std::cerr << "ERROR: Invalid row_
            index." << std::endl; exit(EXIT_FAILURE);}
23         if (y <= 0 || y > c) {std::cerr << "ERROR: Invalid_
            column_index." << std::endl; exit(EXIT_FAILURE);}
24         return data[c * (x - 1) + (y - 1)];
25     }
26
27     double matrix::val (int x, int y) const {
28         if (x <= 0 || x > r) {std::cerr << "ERROR: Invalid row_
            index." << std::endl; exit(EXIT_FAILURE);}
29         if (y <= 0 || y > c) {std::cerr << "ERROR: Invalid_
            column_index." << std::endl; exit(EXIT_FAILURE);}
30         return data[c * (x - 1) + (y - 1)];
31     }
32
33     bool matrix::operator==(const matrix& m) const {
34         if (r != m.rows() || c != m.cols()) {
35             return false;
36         }
37         for (int i = 1; i <= r; i++) {
38             for (int j = 1; j <= c; j++) {
39                 if (val(i, j) != m.val(i, j)) {
40                     return false;
41                 }
42             }
43         }
44         return true;
45     }
46
47     matrix& matrix::operator= (const matrix& m) {
48         if (this != &m) {
49             delete[] data;
50             r = m.rows(); c = m.cols();

```

```

51         data = new double[r * c];
52         for (int i = 0; i < (r * c); i++) {
53             data[i] = m.data[i];
54         }
55     }
56     return *this;
57 }
58
59 matrix matrix::operator+ (const matrix& m) const {
60     if (r != m.rows() || c != m.cols()) {
61         std::cerr << "ERROR: _Addition_size_mismatch."
62             << std::endl;
63         exit(EXIT_FAILURE);
64     }
65     matrix ma(r, c);
66     for (int i = 1; i <= r; i++) {
67         for (int j = 1; j <= c; j++) {
68             ma.val(i, j) = val(i, j) + m.val(i, j);
69         }
70     }
71     return ma;
72 }
73
74 matrix matrix::operator- (const matrix& m) const {
75     if (r != m.rows() || c != m.cols()) {
76         std::cerr << "ERROR: _Subtraction_size_mismatch."
77             << std::endl;
78         exit(EXIT_FAILURE);
79     }
80     matrix ma(r, c);
81     for (int i = 1; i <= r; i++) {
82         for (int j = 1; j <= c; j++) {
83             ma.val(i, j) = val(i, j) - m.val(i, j);
84         }
85     }
86     return ma;
87 }
88
89 matrix matrix::operator* (const double& f) const {
90     matrix ma(r, c);

```

```

89         for (int i = 1; i <= r; i++) {
90             for (int j = 1; j <= c; j++) {
91                 ma.val(i, j) = val(i, j) * f;
92             }
93         }
94         return ma;
95     }
96
97     matrix matrix::operator* (const matrix& m) const {
98         if (c != m.rows()) {
99             std::cerr << "ERROR:_Multiplication_size_
100                 mismatch." << std::endl;
101             exit(EXIT_FAILURE);
102         }
103         matrix ma(r, m.cols());
104         for (int i = 1; i <= ma.rows(); i++) {
105             for (int j = 1; j <= ma.cols(); j++) {
106                 ma.val(i, j) = 0;
107                 for (int k = 1; k <= c; k++) {
108                     ma.val(i, j) += (val(i, k) * m.
109                         val(k, j));
110                 }
111             }
112         }
113         return ma;
114     }
115
116     matrix operator* (const double& f, const matrix& m) {
117         matrix mt = m;
118         return (mt * f);
119     }
120
121     matrix matrix::transpose () const {
122         matrix ma(cols(), rows());
123         for (int i = 1; i <= rows(); i++) {
124             for (int j = 1; j <= cols(); j++) {
125                 ma.val(j, i) = val(i, j);
126             }
127         }
128         return ma;

```

```

127 }
128
129 matrix matrix::gaussElim () const {
130     matrix u = *this;
131     for (int k = 1; k <= u.rows(); k++) {
132         int i = k, flag = 1;
133         while (u.val(i, k) == 0) {
134             if (i < u.rows()) {
135                 i++;
136             } else {
137                 throw std::runtime_error("
                    singular_matrix");
138                 break;
139             }
140         }
141         if (i != k && flag) {
142             double temp;
143             for (int j = 1; j <= u.cols(); j++) {
144                 temp = u.val(i, j);
145                 u.val(i, j) = u.val(k, j);
146                 u.val(k, j) = temp;
147             }
148         }
149         for (int i = k + 1; i <= u.rows(); i++) {
150             double temp;
151             if (u.val(k,k)) {
152                 temp = u.val(i, k) / u.val(k, k
                    );
153             } else {
154                 temp = 0;
155             }
156             for (int j = 1; j <= u.cols(); j++) {
157                 u.val(i, j) = u.val(i, j) - (
                    temp * u.val(k, j));
158             }
159         }
160     }
161     return u;
162 }
163

```

```

164 matrix matrix::inv() const {
165     if (rows() != cols()) {
166         std::cerr << "ERROR:_Can't_invert_of_non-square
           _matrix." << std::endl;
167         exit(EXIT_FAILURE);
168     }
169     matrix temp(rows(),cols());
170     for (int i = 1; i <= rows(); i++) {
171         for (int j = 1; j <= rows(); j++) {
172             temp.val(i, j) = val(i, j);
173         }
174     }
175     matrix ma(rows(), cols()), mx;
176     for (int j = 1; j <= rows(); j++) {
177         matrix mi(rows(), 1);
178         for (int i = 1; i <= rows(); i++) {
179             if (i == j) {
180                 mi.val(i, 1) = 1;
181             } else {
182                 mi.val(i, 1) = 0;
183             }
184         }
185         try {
186             mx = solnLinearEqns(temp, mi);
187         } catch (const std::runtime_error& err) {
188             std::cerr << "ERROR:_Can't_invert_" <<
               err.what() << std::endl;
189             exit(EXIT_FAILURE);
190         }
191         for (int i = 1; i <= rows(); i++) {
192             ma.val(i, j) = mx.val(i, 1);
193         }
194     }
195     return ma;
196 }
197
198 matrix I (int size) {
199     matrix ma(size,size);
200     for (int i = 1; i <= size; i++) {
201         ma.val(i,i) = 1;

```

```

202     }
203     return ma;
204 }
205
206 matrix solnLinearEqns (const matrix& A, const matrix& B) {
207     if (A.rows() != A.cols() || A.cols() != B.rows() || B.
        cols() != 1) {
208         std::cerr << "ERROR:_System_of_linear_equations
            _size_mismatch." << std::endl;
209         exit(EXIT_FAILURE);
210     }
211     matrix comb(A.rows(), A.cols() + 1), m;
212     for (int i = 1; i <= comb.rows(); i++) {
213         for (int j = 1; j <= comb.cols(); j++) {
214             if (j < comb.cols()) {
215                 comb.val(i, j) = A.val(i, j);
216             } else {
217                 comb.val(i, j) = B.val(i, 1);
218             }
219         }
220     }
221     try {
222         m = comb.gaussElim();
223     } catch (const std::runtime_error& err) {
224         throw err;
225     }
226     matrix A_temp(A.rows(), A.cols()), B_temp(B.rows(), B.
        cols()), X(B.rows(), B.cols());
227     for (int i = 1; i <= m.rows(); i++) {
228         for (int j = 1; j <= m.cols(); j++) {
229             if (j < m.cols()) {
230                 A_temp.val(i, j) = m.val(i, j);
231             } else {
232                 B_temp.val(i, 1) = m.val(i, j);
233             }
234         }
235     }
236     for (int i = A_temp.rows(); i > 0; i--) {
237         double sum = 0;
238         for (int j = i; j <= A_temp.cols(); j++) {

```



```

239             sum += A_temp.val(i, j) * X.val(j, 1);
240         }
241         X.val(i, 1) = (B_temp.val(i, 1) - sum) / A_temp
                .val(i, i);
242     }
243     return X;
244 }
245
246 std::vector<matrix> matrix::plu () const {
247     matrix p = I(rows()), l = I(rows());
248     matrix u = *this;
249     for (int k = 1; k <= u.rows(); k++) {
250         int i = k, flag = 1;
251         while (u.val(i, k) == 0) {
252             if (i < u.rows()) {
253                 i++;
254             } else {
255                 flag = 0;
256                 break;
257             }
258         }
259         if (i != k && flag) {
260             double temp;
261             for (int j = 1; j <= u.cols(); j++) {
262                 temp = u.val(i, j);
263                 u.val(i, j) = u.val(k, j);
264                 u.val(k, j) = temp;
265             }
266             for (int j = 1; j < k; j++) {
267                 temp = l.val(i, j);
268                 l.val(i, j) = l.val(k, j);
269                 l.val(k, j) = temp;
270             }
271             for (int j = 1; j <= p.rows(); j++) {
272                 temp = p.val(i, j);
273                 p.val(i, j) = p.val(k, j);
274                 p.val(k, j) = temp;
275             }
276         }
277         for (int i = k + 1; i <= u.rows(); i++) {

```

```

278         double temp;
279         if (u.val(k,k)) {
280             temp = u.val(i, k) / u.val(k, k
                );
281         } else {
282             temp = 0;
283         }
284         for (int j = 1; j <= cols(); j++) {
285             u.val(i, j) = u.val(i, j) - (
                temp * u.val(k, j));
286         }
287         l.val(i,k) = temp;
288     }
289 }
290 std::vector<matrix> va;
291 va.push_back(p.transpose()); va.push_back(l); va.
    push_back(u);
292 return va;
293 }
294
295 matrix matrix::house () const {
296     if (c != 1) {
297         std::cerr << "ERROR:_Householder_size_mismatch.
            " << std::endl;
298         exit(EXIT_FAILURE);
299     }
300     matrix v = *this;
301     v.val(1,1) = 0;
302     double beta, mu, sigma = (v.transpose() * v).val(1,1);
303     v.val(1,1) = 1;
304     if (sigma == 0) {
305         if (val(1,1) < 0) {
306             beta = 2;
307         } else {
308             beta = 0;
309         }
310     } else {
311         mu = sqrt((val(1,1) * val(1,1)) + sigma);
312         if (val(1,1) > 0) {
313             v.val(1,1) = - sigma / (val(1,1) + mu);

```

```

314         } else {
315             v.val(1,1) = val(1,1) - mu;
316         }
317         beta = (2 * v.val(1,1) * v.val(1,1)) / (sigma +
            (v.val(1,1) * v.val(1,1)));
318         v = v * (1.0 / v.val(1,1));
319     }
320     return I(v.rows()) - (beta * v * v.transpose());
321 }
322
323 std::vector<matrix> matrix::qr () const {
324     matrix qa = I(rows()), ra = *this;
325     for (int k = 1; k <= ra.cols(); k++) {
326         matrix v(ra.rows() + 1 - k,1);
327         for (int j = 1; j <= v.rows(); j++) {
328             v.val(j,1) = ra.val(k+j-1,k);
329         }
330         matrix temp = I(ra.rows()), h = v.house();
331         for (int i = 1; i <= h.rows(); i++) {
332             for (int j = 1; j <= h.cols(); j++) {
333                 temp.val(i+k-1,j+k-1) = h.val(i
                    ,j);
334             }
335         }
336         qa = qa * temp.transpose();
337         ra = temp * ra;
338     }
339     std::vector<matrix> va;
340     va.push_back(qa); va.push_back(ra);
341     return va;
342 }
343
344 bool matrix::isUpperTriangular (double threshold) const {
345     if (rows() != cols()) {
346         std::cerr << "ERROR:_Upper_triangularity_cannot
            _be_determined_for_non-square_matrix." <<
            std::endl;
347     }
348     if (rows() < 2 || cols() < 2) {
349         return false;

```

```

350     }
351     for (int i = 2; i <= rows(); i++) {
352         for (int j = 1; j < i; j++) {
353             if (fabs(val(i,j)) > fabs(threshold)) {
354                 return false;
355             }
356         }
357     }
358     return true;
359 }
360
361 matrix matrix::eigVal (double threshold) const {
362     if (rows() != cols()) {
363         std::cerr << "ERROR:_Can't_find_eigen_values_
364             for_non-square_matrix." << std::endl;
365         exit(EXIT_FAILURE);
366     }
367     matrix temp = *this, temp_old;
368     int i = 0;
369     while (!temp.isUpperTriangular(threshold)) {
370         std::vector<matrix> factors = temp.qr();
371         matrix q = factors.at(0);
372         matrix r = factors.at(1);
373         temp_old = temp;
374         temp = r * q;
375         if (temp_old == temp) {
376             temp = r;
377             break;
378         }
379         i++;
380     }
381     matrix ma(rows(), cols());
382     std::cout << i << "_iterations_for_QR_Algorithm" << std
383         ::endl;
384     for(int i = 1; i <= rows(); i++) {
385         ma.val(i,i) = temp.val(i,i);
386     }
387     return ma;

```

```

388 matrix matrix::eigVec (double threshold) const {
389     if (rows() != cols()) {
390         std::cerr << "ERROR:_Can't_find_eigen_values_
           for_non-square_matrix." << std::endl;
391         exit(EXIT_FAILURE);
392     }
393     matrix temp = *this, temp_old, v = I(rows());
394     int i = 0;
395     while (!temp.isUpperTriangular(threshold)) {
396         std::vector<matrix> factors = temp.qr();
397         matrix q = factors.at(0);
398         matrix r = factors.at(1);
399         temp_old = temp;
400         temp = r * q;
401         v = v * q;
402         if (temp_old == temp) {
403             v = q;
404             break;
405         }
406         i++;
407     }
408     std::cout << i << "_iterations_for_QR_Algorithm" << std
           ::endl;
409     return v;
410 }

```

vector.hpp

```

1  #include <vector>
2
3  template <typename T>
4  std::vector<T> operator+ (const std::vector<T>& v, const T& t)
      {
5      std::vector<T> va = v;
6      for (int i = 0; i < v.size(); i++) {
7          va.at(i) = va.at(i) + t;
8      }
9      return va;
10 }
11
12 template <typename T>

```

```

13 std::vector<T> operator- (const std::vector<T>& v, const T& t)
    {
14     std::vector<T> va = v;
15     for (int i = 0; i < v.size(); i++) {
16         va.at(i) = va.at(i) - t;
17     }
18     return va;
19 }
20
21 template <typename T>
22 T mean (const std::vector<T>& v) {
23     T ta = v.at(0);
24     for (int i = 1; i < v.size(); i++) {
25         ta = v.at(i) * (1.0 / (i + 1)) + ta * (1.0 * i
                / (i + 1));
26     }
27     return ta;
28 }

```

A.2 Rigid Library

rigid.hpp

```

1  #ifndef RIGID_HPP
2  #define RIGID_HPP
3
4  #include <iostream>
5  #include "matrix.hpp"
6
7  // VECTORS
8
9  class vec3 {
10     double x_comp = 0, y_comp = 0, z_comp = 0;
11 public:
12     // Constructor
13     vec3 () = default;
14     vec3 (double x, double y, double z) : x_comp(x), y_comp
        (y), z_comp(z) {}
15     vec3 (const vec3& v) : x_comp(v.x()), y_comp(v.y()),
        z_comp(v.z()) {}

```

```

16         // Destructor
17         ~vec3 () = default;
18         // Essential Functions
19         double x() const {return x_comp;}
20         double y() const {return y_comp;}
21         double z() const {return z_comp;}
22         // Operators
23         vec3& operator= (const vec3&);
24         vec3 operator+ (const vec3&) const;
25         vec3 operator- (const vec3&) const;
26         vec3 operator* (const double&) const;
27         // Other Functions
28         double norm() const;
29     };
30
31     std::ostream& operator<< (std::ostream&, const vec3&);
32
33     vec3 operator* (const double&, const vec3&);
34
35     vec3 operator* (const matrix&, const vec3&);
36
37     double ip (const vec3&, const vec3&);
38
39     vec3 cp (const vec3&, const vec3&);
40
41     matrix op (const vec3&, const vec3&);
42
43     // QUATERNIONS
44
45     class quaternion {
46         double q_0 = 1, q_i = 0, q_j = 0, q_k = 0;
47     public:
48         // Constructor
49         quaternion () = default;
50         quaternion (double w, double x, double y, double z) :
51             q_0(w), q_i(x), q_j(y), q_k(z) {}
52         quaternion (const quaternion& q) : q_0(q.q0()), q_i(q.
53             qi()), q_j(q.qj()), q_k(q.qk()) {}
54         // Destructor
55         ~quaternion () = default;

```

```

54      // Essential Functions
55      double q0() const {return q_0;}
56      double qi() const {return q_i;}
57      double qj() const {return q_j;}
58      double qk() const {return q_k;}
59      // Operators
60      quaternion& operator= (const quaternion&);
61      quaternion operator+ (const quaternion&) const;
62      quaternion operator- (const quaternion&) const;
63      quaternion operator* (const double&) const;
64      quaternion operator* (const quaternion&) const;
65      // Other Functions
66      quaternion conjugate () const;
67      double norm () const;
68      matrix dcm () const;
69      vec3 L (const vec3&) const;
70      std::vector<vec3> L (const std::vector<vec3>&) const;
71  };
72
73  std::ostream& operator<< (std::ostream&, const quaternion&);
74
75  quaternion operator* (const double&, const quaternion&);
76
77  // RIGID BODY
78
79  typedef struct {
80      double mass;
81      matrix inertia;
82      vec3 position;
83      quaternion orientation;
84      vec3 lin_vel, ang_vel;
85      vec3 force, torque;
86  } rigid;
87
88  void odeStep (rigid& rb, const double& h);
89
90  #endif

```

rigid.cpp

```

1  #include "rigid.hpp"

```



```

2  #include <iostream>
3  #include <cmath>
4
5  // VECTORS
6
7  vec3& vec3::operator= (const vec3& v) {
8      if (this != &v) {
9          x_comp = v.x();
10         y_comp = v.y();
11         z_comp = v.z();
12     }
13     return *this;
14 }
15
16 vec3 vec3::operator+ (const vec3& v) const {
17     vec3 va(x() + v.x(), y() + v.y(), z() + v.z());
18     return va;
19 }
20
21 vec3 vec3::operator- (const vec3& v) const {
22     vec3 va(x() - v.x(), y() - v.y(), z() - v.z());
23     return va;
24 }
25
26 vec3 vec3::operator* (const double& f) const {
27     vec3 va(x() * f, y() * f, z() * f);
28     return va;
29 }
30
31 double vec3::norm () const {
32     vec3 v = *this;
33     return sqrt(ip(v,v));
34 }
35
36 std::ostream& operator<< (std::ostream& os, const vec3& v) {
37     os << v.x() << "\t" << v.y() << "\t" << v.z();
38     return os;
39 }
40
41 vec3 operator* (const double& f, const vec3& v) {

```

```

42     vec3 va(f * v.x(), f * v.y(), f * v.z());
43     return va;
44 }
45
46 vec3 operator* (const matrix& m, const vec3& v) {
47     if (m.rows() != 3 || m.cols() != 3) {
48         std::cerr << "ERROR: Matrix cannot be_
49             multiplied_to_vector_due_to_size_mismatch."
50             << std::endl;
51         exit(EXIT_FAILURE);
52     }
53     matrix mv(3,1), ma; mv.val(1,1) = v.x(); mv.val(2,1) =
54         v.y(); mv.val(3,1) = v.z();
55     ma = m * mv;
56     vec3 va(ma.val(1,1), ma.val(2,1), ma.val(3,1));
57     return va;
58 }
59
60 double ip (const vec3& v1, const vec3& v2) {
61     return ((v1.x() * v2.x()) + (v1.y() * v2.y()) + (v1.z()
62         * v2.z()));
63 }
64
65 vec3 cp (const vec3& v1, const vec3& v2) {
66     vec3 va((v1.y() * v2.z() - v1.z() * v2.y()), (v1.z() *
67         v2.x() - v1.x() * v2.z()), (v1.x() * v2.y() - v1.y()
68         * v2.x()));
69     return va;
70 }
71
72 matrix op (const vec3& v1, const vec3& v2) {
73     matrix m1(3,1), m2(1,3), ma;
74     m1.val(1,1) = v1.x(); m1.val(2,1) = v1.y(); m1.val(3,1)
75         = v1.z();
76     m2.val(1,1) = v2.x(); m2.val(1,2) = v2.y(); m2.val(1,3)
77         = v2.z();
78     ma = m1 * m2;
79     return ma;
80 }

```

```

74 // QUATERNIONS
75
76 quaternion& quaternion::operator= (const quaternion& q) {
77     if (this != &q) {
78         q_0 = q.q0();
79         q_i = q.qi();
80         q_j = q.qj();
81         q_k = q.qk();
82     }
83     return *this;
84 }
85
86 quaternion quaternion::operator+ (const quaternion& q) const {
87     quaternion qa(q0() + q.q0(), qi() + q.qi(), qj() + q.qj
88         (), qk() + q.qk());
89     return qa;
90 }
91
92 quaternion quaternion::operator- (const quaternion& q) const {
93     quaternion qa(q0() - q.q0(), qi() - q.qi(), qj() - q.qj
94         (), qk() - q.qk());
95     return qa;
96 }
97
98 quaternion quaternion::operator* (const double& f) const {
99     quaternion qa(q0() * f, qi() * f, qj() * f, qk() * f);
100     return qa;
101 }
102
103 quaternion quaternion::operator* (const quaternion& q) const {
104     matrix m(4,4), mq(4,1), ma;
105     mq.val(1,1) = q.q0(); mq.val(2,1) = q.qi(); mq.val(3,1)
106         = q.qj(); mq.val(4,1) = q.qk();
107     m.val(1,1) = q0(); m.val(1,2) = -qi(); m.val(1,3) = -qj
108         (); m.val(1,4) = -qk();
109     m.val(2,1) = qi(); m.val(2,2) = q0(); m.val(2,3) = -qk
110         (); m.val(2,4) = qj();
111     m.val(3,1) = qj(); m.val(3,2) = qk(); m.val(3,3) = q0()
112         ; m.val(3,4) = -qi();
113     m.val(4,1) = qk(); m.val(4,2) = -qj(); m.val(4,3) = qi

```

```

        (); m.val(4,4) = q0();
108     ma = m * mq;
109     quaternion qa(ma.val(1,1), ma.val(2,1), ma.val(3,1), ma
        .val(4,1));
110     return qa;
111 }
112
113 quaternion quaternion::conjugate() const {
114     quaternion qa(q0(), -qi(), -qj(), -qk());
115     return qa;
116 }
117
118 double quaternion::norm () const {
119     quaternion q = *this, qa;
120     return sqrt((q * q.conjugate()).q0());
121 }
122
123 matrix quaternion::dcm () const {
124     matrix ma(3,3);
125     ma.val(1,1) = (2 * q0() * q0()) + (2 * qi() * qi()) -
        1;
126     ma.val(1,2) = (2 * qi() * qj()) - (2 * q0() * qk());
127     ma.val(1,3) = (2 * qi() * qk()) + (2 * q0() * qj());
128     ma.val(2,1) = (2 * qi() * qj()) + (2 * q0() * qk());
129     ma.val(2,2) = (2 * q0() * q0()) + (2 * qj() * qj()) -
        1;
130     ma.val(2,3) = (2 * qj() * qk()) - (2 * q0() * qi());
131     ma.val(3,1) = (2 * qi() * qk()) - (2 * q0() * qj());
132     ma.val(3,2) = (2 * qj() * qk()) + (2 * q0() * qi());
133     ma.val(3,3) = (2 * q0() * q0()) + (2 * qk() * qk()) -
        1;
134     return ma;
135 }
136
137 vec3 quaternion::L (const vec3& v) const {
138     quaternion q = *this, qv(0,v.x(),v.y(),v.z()), qa;
139     qa = q * qv * q.conjugate();
140     vec3 va(qa.qi(), qa.qj(), qa.qk());
141     return va;
142 }

```

```

143
144 std::vector<vec3> quaternion::L (const std::vector<vec3>& v)
    const {
145     std::vector<vec3> va;
146     for (int i = 0; i < v.size(); i++) {
147         va.push_back(L(v.at(i)));
148     }
149     return va;
150 }
151
152 std::ostream& operator<< (std::ostream& os, const quaternion& q
    ) {
153     os << q.q0() << "\t" << q.qi() << "\t" << q.qj() << "\t"
        << q.qk();
154     return os;
155 }
156
157 quaternion operator* (const double& f, const quaternion& q) {
158     quaternion va(f * q.q0(), f * q.qi(), f * q.qj(), f * q
        .qk());
159     return va;
160 }
161
162 void odeStep (rigid& rb, const double& h) {
163     rb.lin_vel = rb.lin_vel + (h * rb.force * (1.0 / rb.
        mass));
164     rb.position = rb.position + (h * rb.lin_vel);
165     {
166         vec3 k1_ang = h * rb.inertia.inv() * (rb.torque
            - cp(rb.ang_vel,rb.inertia * rb.ang_vel));
167         vec3 k2_ang = h * rb.inertia.inv() * (rb.torque
            - cp(rb.ang_vel + 0.5 * k1_ang,rb.inertia *
            (rb.ang_vel + 0.5 * k1_ang)));
168         vec3 k3_ang = h * rb.inertia.inv() * (rb.torque
            - cp(rb.ang_vel + 0.5 * k2_ang,rb.inertia *
            (rb.ang_vel + 0.5 * k2_ang)));
169         vec3 k4_ang = h * rb.inertia.inv() * (rb.torque
            - cp(rb.ang_vel + k3_ang,rb.inertia * (rb.
            ang_vel + k3_ang)));
170         rb.ang_vel = rb.ang_vel + ((k1_ang + (2.0 *

```

```

        k2_ang) + (2.0 * k3_ang) + k4_ang) * (1.0 /
        6.0));
171     }
172     {
173         quaternion omega(0,rb.ang_vel.x(),rb.ang_vel.y
            (),rb.ang_vel.z());
174         quaternion k1_ori = h * 0.5 * rb.orientation *
            omega;
175         quaternion k2_ori = h * 0.5 * (rb.orientation +
            0.5 * k1_ori) * omega;
176         quaternion k3_ori = h * 0.5 * (rb.orientation +
            0.5 * k2_ori) * omega;
177         quaternion k4_ori = h * 0.5 * (rb.orientation +
            k3_ori) * omega;
178         rb.orientation = rb.orientation + ((k1_ori +
            (2.0 * k2_ori) + (2.0 * k3_ori) + k4_ori) *
            (1.0 / 6.0));
179     }
180     return ;
181 }

```

A.3 Pose Library

pose.hpp

```

1  #ifndef POSE_HPP
2  #define POSE_HPP
3
4  #include "rigid.hpp"
5
6  struct pose {
7      vec3 position;
8      quaternion orientation;
9  };
10
11 std::vector<vec3> pcImport (std::ifstream&);
12
13 // void pcExport (std::ifstream&, const std::vector<vec3>&);
14
15 std::vector<vec3> pcWhiteGaussianNoise (const std::vector<vec3>

```

```

    >&, const double&, const double&);
16
17 pose poseEst (const std::vector<vec3>&, const std::vector<vec3
    >&, const double&);
18
19 pose icp (const std::vector<vec3>&, const std::vector<vec3>&,
    const double&, const double&);
20
21 struct pcobj {
22     rigid rb;
23     std::vector<vec3> pc;
24 };
25
26 class sensor {
27 public:
28     rigid rb;
29     double hfov, vfov, rmin, rmax;
30     // Constructor
31     sensor () = default;
32     // Destructor
33     ~sensor () = default;
34     // Functions
35     bool inFov (const vec3&);
36     std::vector<vec3> scan (const pcobj&);
37 };
38
39 #endif

```

pose.cpp

```

1 #include "pose.hpp"
2 #include "vector.hpp"
3 #include <iostream>
4 #include <fstream>
5 #include <sstream>
6 #include <random>
7 #include <cmath>
8
9 std::vector<vec3> pcImport (std::ifstream& fr) {
10     if (!fr.is_open()) {
11         std::cerr << "ERROR: Cannot open the pointcloud

```

```

        _file!" << std::endl;
12         exit(EXIT_FAILURE);
13     }
14     std::vector<vec3> va;
15     std::string line;
16     while (std::getline(fr, line)) {
17         std::istringstream stream(line);
18         double x, y, z;
19         if (!(stream >> x >> y >> z)) {
20             std::cerr << "ERROR:_Invalid_data_in_
                point_cloud_file." << std::endl;
21             exit(EXIT_FAILURE);
22         }
23         vec3 v_temp(x,y,z);
24         va.push_back(v_temp);
25     }
26     return va;
27 }
28
29 std::vector<vec3> pcWhiteGaussianNoise (const std::vector<vec3
    >& pc, const double& mean, const double& stdev) {
30     std::random_device rand;
31     std::mt19937 noise(rand());
32     std::normal_distribution<double> gaussian(mean, stdev);
33     std::vector<vec3> pca(pc.size());
34     for (int i = 0; i < pc.size(); i++) {
35         vec3 n(gaussian(noise), gaussian(noise), gaussian
            (noise));
36         pca.at(i) = pc.at(i) + n;
37     }
38     return pca;
39 }
40
41 pose poseEst (const std::vector<vec3>& ref, const std::vector<
    vec3>& img, const double& threshold) {
42     if(ref.size() != img.size()) {
43         std::cerr << "ERROR:_Pose_estimate_size_
            mismatch." << std::endl;
44         exit(EXIT_FAILURE);
45     }

```



```

46     vec3 ref_c = mean(ref);
47     vec3 img_c = mean(img);
48     matrix covariance(3,3);
49     for (int i = 0; i < ref.size(); i++) {
50         covariance = ((covariance * (1.0 * i)) + op(img
                    .at(i),ref.at(i))) * (1.0 / (i + 1));
51     }
52     covariance = covariance - op(img_c,ref_c);
53     matrix q(4,4), eigVec;
54     q.val(1,1) = covariance.val(1,1) + covariance.val(2,2)
        + covariance.val(3,3);
55     q.val(1,2) = covariance.val(2,3) - covariance.val(3,2);
56     q.val(1,3) = covariance.val(3,1) - covariance.val(1,3);
57     q.val(1,4) = covariance.val(1,2) - covariance.val(2,1);
58     q.val(2,1) = covariance.val(2,3) - covariance.val(3,2);
59     q.val(2,2) = covariance.val(1,1) - covariance.val(2,2)
        - covariance.val(3,3);
60     q.val(2,3) = covariance.val(1,2) + covariance.val(2,1);
61     q.val(2,4) = covariance.val(3,1) + covariance.val(1,3);
62     q.val(3,1) = covariance.val(3,1) - covariance.val(1,3);
63     q.val(3,2) = covariance.val(1,2) + covariance.val(2,1);
64     q.val(3,3) = covariance.val(2,2) - covariance.val(1,1)
        - covariance.val(3,3);
65     q.val(3,4) = covariance.val(2,3) + covariance.val(3,2);
66     q.val(4,1) = covariance.val(1,2) - covariance.val(2,1);
67     q.val(4,2) = covariance.val(3,1) + covariance.val(1,3);
68     q.val(4,3) = covariance.val(2,3) + covariance.val(3,2);
69     q.val(4,4) = covariance.val(3,3) - covariance.val(1,1)
        - covariance.val(2,2);
70     eigVec = q.eigVec(threshold);
71     quaternion qa(eigVec.val(1,1),eigVec.val(2,1),eigVec.
        val(3,1),eigVec.val(4,1));
72     vec3 va = ref_c - qa.L(img_c);
73     pose pa; pa.position = va; pa.orientation = qa;
74     return pa;
75 }
76
77 pose icp (const std::vector<vec3>& ref, const std::vector<vec3
    >& img, const double& threshold, const double& qr) {
78     int iter = 0; double err, err_old;

```

```

79     pose estimate;
80     std::vector<vec3> temp_img = img;
81     while (iter == 0 || (err_old - err) > threshold) {
82         std::vector<vec3> cpy(img.size(),img.at(0));
83         err_old = err; err = 0;
84         for (int i = 0; i < temp_img.size(); i++) {
85             for (int j = 0; j < ref.size(); j++) {
86                 if ((ref.at(j) - temp_img.at(i)
87                     ).norm() < (cpy.at(i) -
88                         temp_img.at(i)).norm()) {
89                     cpy.at(i) = ref.at(j);
90                 }
91             }
92             estimate = poseEst(ref,temp_img,qr);
93             temp_img = estimate.orientation.L(ref);
94             temp_img = temp_img + estimate.position;
95             for (int i = 0; i < temp_img.size(); i++) {
96                 double et = (cpy.at(i) - temp_img.at(i)
97                     ).norm();
98                 err = (err * i / (i + 1.0)) + (et * et
99                     / (i + 1.0));
100             }
101             iter++;
102         }
103     }
104     return estimate;
105 }
106
107 bool sensor::inFov (const vec3& v) {
108     vec3 vt = v - rb.position;
109     vt = rb.orientation.conjugate().L(vt);
110     if (vt.norm() > rmax || vt.norm() < rmin) {
111         return false;
112     }
113     matrix mxy(3,2); mxy.val(1,1) = 1; mxy.val(2,2) = 1;
114     matrix mxz(3,2); mxz.val(1,1) = 1; mxz.val(3,2) = 1;
115     vec3 vxy = (mxy * mxy.transpose()) * vt;
116     vec3 vxz = (mxz * mxz.transpose()) * vt;
117     double ch = vxy.x() / vxy.norm(); double cv = vxz.x() /
118         vxz.norm();

```

```

114         if (ch < cos(hfov / 2.0) || cv < cos(vfov / 2.0)) {
115             return false;
116         }
117         return true;
118     }
119
120     std::vector<vec3> sensor::scan(const pcobj& target) {
121         std::vector<vec3> pca, pct;
122         pct = target.rb.orientation.L(target.pc);
123         pct = pct + target.rb.position;
124         for (int i = 0; i < pct.size(); i++) {
125             if (inFov(pct.at(i))) {
126                 pca.push_back(rb.orientation.L(pct.at(i)
127                     ) - rb.position));
128             }
129         }
130         return pca;
131     }

```

A.4 20000 Second Simulation

rk4.hpp

```

1  #include <iostream>
2  #include <fstream>
3  #include <cmath>
4  #include "rigid.hpp"
5
6  void print(const matrix&);
7
8  int main () {
9      matrix m(3,3); double h = 0.01;
10     m.val(1,1) = 25; m.val(1,2) = -0.5; m.val(1,3) = 1.5;
11     m.val(2,1) = -0.5; m.val(2,2) = 30; m.val(2,3) = 2;
12     m.val(3,1) = 1.5; m.val(3,2) = 2; m.val(3,3) = 28;
13     vec3 omega(M_PI/180,-M_PI/180,M_PI/180);
14     quaternion q(0,0,0,1);
15     rigid rb;
16     rb.inertia = m;
17     rb.ang_vel = omega;

```

```

18     rb.orientation = q;
19     std::ofstream fw1("rk4_omega.dat");
20     std::ofstream fw2("rk4_quat.dat");
21     std::ofstream fw3("rk4_euler.dat");
22     for (int i = 0; i <= 2000000; i++) {
23         odeStep(rb,h);
24         matrix dircos = rb.orientation.dcm();
25         fw1 << (h * i) << "\t" << rb.ang_vel.x() * (180 /
            / M_PI) << "\t" << rb.ang_vel.y() * (180 /
            M_PI) << "\t" << rb.ang_vel.z() * (180 /
            M_PI) << "\t" << (rb.inertia * rb.ang_vel).
            norm() << std::endl;
26         fw2 << (h * i) << "\t" << rb.orientation.q0()
            << "\t" << rb.orientation.qi() << "\t" << rb
            .orientation.qj() << "\t" << rb.orientation.
            qk() << "\t" << rb.orientation.norm() << std
            ::endl;
27         fw3 << (h * i) << "\t" << atan2(dircos.val(1,2)
            ,dircos.val(1,1)) * (180 / M_PI) << "\t" <<
            asin(-dircos.val(1,3)) * (180 / M_PI) << "\t
            " << atan2(dircos.val(2,3),dircos.val(3,3))
            * (180 / M_PI) << std::endl;
28     }
29     fw1.close(); fw2.close(); fw3.close();
30     return 0;
31 }
32
33 void print(const matrix& m) {
34     for (int i = 1; i <= m.rows(); i++) {
35         for (int j = 1; j <= m.cols(); j++) {
36             std::cout << m.val(i,j) << "\t";
37         }
38         std::cout << std::endl;
39     }
40 }

```

A.5 Pose Estimation

estimation.hpp

```

1  #include <cmath>
2  #include <fstream>
3  #include "pose.hpp"
4  #include "vector.hpp"
5
6  void kalman (matrix&, matrix&, const vec3&, const vec3&, const
    matrix&, const matrix&, const double&);
7
8  int main () {
9      double t = 0, h = 0.1, t_max = 20, noise_stdev = 5,
        cutoff = 0.1 * M_PI;
10     pcobj target, reference;
11     pose measured; quaternion old, filtered, old_filt,
        omega;
12     // Init target
13     std::ifstream fr("original.dat");
14     target.pc = pcImport(fr); fr.close();
15     target.rb.position = mean(target.pc);
16     target.pc = target.pc - target.rb.position;
17     target.rb.mass = 1; target.rb.inertia = I(3);
18     target.rb.lin_vel = vec3(6,4,2); target.rb.ang_vel =
        vec3(0,0,-M_PI/180);
19     // Init reference
20     reference.pc = target.pc;
21     // Init Kalman filter
22     matrix state(6,1), cov = 10000 * I(6), noise_cov =
        noise_stdev * I(3);
23     // Init output files
24     std::ofstream fposiact("posi_act.dat"), forieact("
        orie_act.dat"), fveloact("velo_act.dat"), fomegact("
        omeg_act.dat"), fposimea("posi_mea.dat"), foriemia("
        orie_mea.dat"), fposiest("posi_est.dat"), fveloest("
        velo_est.dat"), forieest("orie_est.dat"), fomegest("
        omeg_est.dat");
25     // Simulation
26     while (t < t_max) {
27         matrix dcm_a = target.rb.orientation.dcm(),
            dcm_m, dcm_e;
28         old = measured.orientation; old_filt = filtered
            ;

```

```

29      fposiact << t << "\t" << target.rb.position <<
        std::endl;
30      forieact << t << "\t" << atan2(dcm_a.val(1,2),
        dcm_a.val(1,1)) * (180 / M_PI) << "\t" <<
        asin(-dcm_a.val(1,3)) * (180 / M_PI) << "\t"
        << atan2(dcm_a.val(2,3),dcm_a.val(3,3)) *
        (180 / M_PI) << std::endl;
31      fveloact << t << "\t" << target.rb.lin_vel <<
        std::endl;
32      fomegact << t << "\t" << target.rb.ang_vel *
        (180 / M_PI) << std::endl;
33      measured = icp(pcWhiteGaussianNoise(target.rb.
        orientation.L(target.pc)+target.rb.position
        ,0,noise_stdev),reference.pc,0.01,0.01);
34      dcm_m = measured.orientation.dcm();
35      fposimea << t << "\t" << measured.position <<
        std::endl;
36      foriemea << t << "\t" << atan2(dcm_m.val(1,2),
        dcm_m.val(1,1)) * (180 / M_PI) << "\t" <<
        asin(-dcm_m.val(1,3)) * (180 / M_PI) << "\t"
        << atan2(dcm_m.val(2,3),dcm_m.val(3,3)) *
        (180 / M_PI) << std::endl;
37      kalman(state,cov,measured.position,vec3(),
        matrix(6,6),noise_cov,h);
38      fposiest << t << "\t" << vec3(state.val(1,1),
        state.val(2,1),state.val(3,1)) << std::endl;
39      fveloest << t << "\t" << vec3(state.val(4,1),
        state.val(5,1),state.val(6,1)) << std::endl;
40      if (t < h) {
41          filtered = measured.orientation;
42      } else {
43          filtered = (((measured.orientation +
        old) * (cutoff * h)) + (filtered *
        (2 - cutoff * h))) * (1 / (2 +
        cutoff * h));
44      }
45      omega = (filtered - old_filt) * (2 / h);
46      dcm_e = filtered.dcm();
47      forieest << t << "\t" << atan2(dcm_e.val(1,2),
        dcm_e.val(1,1)) * (180 / M_PI) << "\t" <<

```

```

        asin(-dcm_e.val(1,3)) * (180 / M_PI) << "\t"
        << atan2(dcm_e.val(2,3),dcm_e.val(3,3)) *
        (180 / M_PI) << std::endl;
48     fomegest << t << "\t" << vec3(omega.qi(),omega.
        qj(),omega.qk()) * (180 / M_PI) << std::endl
        ;
49     odeStep(target.rb,h); t += h;
50 }
51 fposiact.close(); forieact.close(); fveloact.close();
    fomegact.close(); fposimea.close(); foriemea.close()
    ; fposiest.close(); fveloest.close(); forieest.close
    (); fomegest.close();
52     return 0;
53 }
54
55 void kalman (matrix& state, matrix& cov, const vec3& measured,
    const vec3& input, const matrix& nq, const matrix& nr, const
    double& h) {
56     matrix ka = I(6), kb(6,3), kh(3,6), kgain, z(3,1), u
        (3,1); ka.val(1,4) = h; ka.val(2,5) = h; ka.val(3,6)
        = h;
57     kb.val(1,1) = h * h / 2; kb.val(2,2) = h * h / 2; kb.
        val(3,3) = h * h / 2; kb.val(4,1) = h; kb.val(5,2) =
        h; kb.val(6,3) = h;
58     kh.val(1,1) = 1; kh.val(2,2) = 1; kh.val(3,3) = 1; u.
        val(1,1) = input.x(); u.val(2,1) = input.y(); u.val
        (3,1) = input.z();
59     z.val(1,1) = measured.x(); z.val(2,1) = measured.y(); z
        .val(3,1) = measured.z();
60     matrix state_predicted = (ka * state) + (kb * u);
        matrix cov_predicted = (ka * cov * ka.transpose()) +
        nq;
61     kgain = cov_predicted * kh.transpose() * ((kh *
        cov_predicted * kh.transpose()) + nr).inv();
62     state = state_predicted + (kgain * (z - kh *
        state_predicted));
63     cov = (I(6) - (kgain * kh)) * cov_predicted;
64     return;
65 }

```

A.6 Sensor Workflow

simulation.hpp

```
1 #include <iostream>
2 #include <fstream>
3 #include <cmath>
4 #include "pose.hpp"
5 #include "vector.hpp"
6
7 int main () {
8     bool dataflag = false, estflag = false, lockflag =
9         false;
10    double h = 0.1, t = 0, t_max = 450, threshold = 0.01;
11    pose measured; quaternion old;
12    vec3 omega_ref(0,0,M_PI/180), omega;
13    std::vector<vec3> img, img_old;
14    // Init lidar and target
15    sensor lidar; pcobj target;
16    std::ifstream fr("original.dat");
17    target.pc = pcImport(fr); fr.close();
18    target.rb.position = mean(target.pc);
19    target.pc = target.pc - target.rb.position;
20    target.rb.mass = 1; target.rb.inertia = I(3); target.rb
21        .ang_vel = vec3(0,0,-M_PI/180);
22    lidar.hfov = M_PI / 3.0; lidar.vfov = M_PI / 2.0; lidar
23        .rb.mass = 1;
24    lidar.rmin = 0; lidar.rmax = 500; lidar.rb.inertia = I
25        (3);
26    std::ofstream fw("simu_omega.dat");
27    while (t < t_max) {
28        img_old = img;
29        img = lidar.scan(target);
30        if (img.size() > 2) {
31            dataflag = true;
32            // img = pcWhiteGaussianNoise(img,0,5);
33            std::cout << t << "\t" ;
34            if (!lockflag && fabs(mean(img).y()) <
35                1) {
36                lockflag = true;
```



```

32         }
33     } else {
34         dataflag = false;
35         lockflag = false;
36     }
37     if (dataflag && (img.size() == img_old.size()))
38     {
39         estflag = true;
40     } else {
41         estflag = false;
42         lockflag = false;
43     }
44     if (dataflag) {
45         old = measured.orientation;
46         if (estflag) {
47             measured = icp(img, img_old,
48                             threshold, threshold);
49             quaternion omega_temp = (2.0 /
50                                     h) * old.conjugate() * (
51                                     quaternion() - measured.
52                                     orientation).conjugate() *
53                                     old;
54             omega = vec3(omega_temp.qi(),
55                           omega_temp.qj(), omega_temp.
56                           qk());
57             omega = omega - lidar.rb.
58                           ang_vel;
59         }
60         if (lockflag) {
61             lidar.rb.torque = (omega -
62                               lidar.rb.ang_vel);
63         }
64     } else {
65         lidar.rb.torque = omega_ref - lidar.rb.
66                               ang_vel;
67     }
68     fw << t << "\t" << omega.z() * (180 / M_PI) <<
69         "\t" << target.rb.ang_vel.z() * (180 / M_PI)
70         << "\t" << lidar.rb.ang_vel.z() * (180 /
71         M_PI) << "\t" << dataflag + estflag +

```

```

        lockflag << std::endl;
58     if (t > 225 && t < 275) {
59         lockflag = false;
60         omega_ref = vec3(0,0,M_PI/90);
61         lidar.rb.torque = omega_ref - lidar.rb.
            ang_vel;
62     }
63     target.rb.torque = (M_PI / 360) * sin(t / 2) *
        (target.rb.inertia * vec3(0,0,1));
64     odeStep(target.rb,h); odeStep(lidar.rb,h);
65     t += h;
66 }
67 fw.close();
68 return 0;
69 }

```